IoT AND MACHINE LEARNING BASED SMART HEALTH MONITORING SYSTEM

A Major Project Report

Submitted to the Faculty of Engineering of

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA, KAKINADA

In partial fulfillment of the requirements

for the Award of Degree of

Bachelor of Technology
in
Electronics and Communication Engineering

Submitted by
SIDAGAM GEETHIKA (22HP1A0468)

Under the Esteemed Guidance of
Mr. P. BOSE BABU, M. Tech., (Ph.D)

Assistant Professor

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

ANDHRA LOYOLA INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Approved by A.I.C.T.E & Affiliated to JNTUK, Kakinada)

An ISO 9001:2015 certified Institution

Vijayawada – 520008

Andhra Pradesh

2022-2026

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

ANDHRA LOYOLA INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Approved by A.I.C.T.E & Affiliated to JNTUK, Kakinada)

An ISO 9001:2015 certified Institution

Vijayawada – 520008

Andhra Pradesh

2022-2026

Certificate

This is to certify that the Project report entitled "**IoT and Machine Learning Based Smart Health Monitoring System"** is a bonafide record of work carried out by **SIDAGAM GEETHIKA (22HP1A0468)** under the guidance of **Mr. P.BOSE BABU, M.Tech.,(Ph.D)** submitted in partial fulfillment of the requirements for the award of degree of **BACHELOR OF TECHNOLOGY** in **ELECTRONICS AND COMMUNICATION ENGINEERING, JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA, KAKINADA** during the academic year **2025-2026**.

(Guide)	**(Head of Department)**
Mr. P. BOSE BABU, M.Tech., (Ph. D)	**Dr. T. Lakshmi Narayana**

(EXTERNAL EXAMINER)

ACKNOWLEDGEMENT

Apart from the efforts of us, the success of any work depends largely on the encouragement and guidelines of many others. We take this opportunity to express our gratitude to the people who have been instrumental in the successful completion of this Project report.

We deeply indebted to our guide **Mr. P. BOSE BABU, M.Tech., (Ph.D.) Assistant Professor, Department of ECE** and it is a great pleasure to express our thanks to her for the encouragement and guidance throughout the course of this Work, with valuable suggestions and ideas that provided the platform to the whole project work. We look at her with great respect for her profound knowledge and relentless pursuit for perfection. Without her encouragement and guidance this project work would not have materialized.

Our sincere thanks to **Dr. T. Lakshmi Narayana, Head of the Department of ECE**, who modeled us both technically and morally for achieving greater success.

We would like to thank our principal of ALIET, **Dr. O. MAHESH** for his valuable statement and suggestion.

We would also like to whole heartedly thank **Rev. Fr. Dr. B. JOJI REDDY S.J., DIRECTOR** and **Rev. Fr.Dr. S. EMMANUEL S.J., Asst. Director ALIET** for helping us a lot during the part of the B.Tech course.

Finally, we owe a lot to our family members, who have been backing throughout our lives, who have been a constant sense of inspiration and whose blessings made me achieve what we applied for.

by

SIDAGAM GEETHIKA 22HP1A0468

CONTENTS

	Pg. No.
List of Figures	i
List of Tables	ii
Nomenclature	iii
Abstract	vi
CHAPTER1: Introduction	**1**
1.1 Motivation of the Research Topic	2
1.2 Objective of the Work	2
1.3 Internet of Things (IoT)	3
1.4 Machine Learning	7
1.5 Overview of the Work	7
1.6 Definition of Health Monitoring System	10
1.7 History of Health Monitoring System	11
CHAPTER2: Literature Survey	**12-15**
CHAPTER3: Patient Health Monitoring System in IoT With ML	**16**
3.1 Aim of Patient Health Monitoring System	17
3.2 Scope of Patient Health Monitoring System	18
CHAPTER4: PROPOSED SYSTEM DESIGN	**20**
4.1 Block Diagram	21
4.2 Flow Chart	22
4.3 Circuit Diagram	24
CHAPTER5: Work Implementation	**26**
5.1 Existing System	27

5.2 Proposed System 27

5.3 Work Implementation 28

CHAPTER6:Hardware Requirement and Pins Specifications 31

6.1GPRS Module 32

6.2 ESP32 33

6.3 Max30102 34

6.4 DS18B20 35

6.5 DHT11 36

6.6 LCD Display 36

CHAPTER7: Software Requirements 38

7.1 Arduino IDE 39

7.2 Installation of ESP32 in Arduino IDE 45

7.3 ThingSpeak Iot Platform 48

7.4 Installation and Running of Python Application 49

CHAPTER8: Results and Discussions 53

8.1 Testing by using LCD 54

8.2 Testing by THINGSSPEAK 55

8.3 Testing by ML Model 56

8.4 Alert SMS 57

CHAPTER9: Advantages And Applications 58-60

Conclusion 61

Future Scope 62

References 63

LIST OF FIGURES

Fig. No	Name of the Figure	Pg. No
1.1	IoT Architecture	4
4.1	Block Diagram of health monitoring system	21
4.2	Flow Chart of health monitoring system	23
4.2	Circuit Diagram of health monitoring system	24
5.1	Work Implementation Flow	29
6.1	GPRS Module	32
6.2	ESP32	33
6.3	Max30102	34
6.4	DS18B20	35
6.5	DHT11	36
6.5	LCD Display	36
7.2.1	Configuration of Additional Board Manager URL for ESP32 in Arduino IDE	46
7.2.2	Accessing the Boards Manager in Arduino Software	46
7.2.3	ESP32 Board Installation Via Arduino Boards Manager	47
7.2.4	conformation of ESP32 Installation in Board Manager	47
7.4.1	Python Installation Setup Window	49
7.4.2	Command Prompt	49
8.1.1	Hardware Setup of Smart Health Monitoring System	54
8.1.2	LCD Display – Alert Notification	54
8.1.3	LCD Display – Real-Time Health Parameters	55
8.2	ThingSpeak Cloud Data Visualization	55
8.3	Patients Condition Checking through ML Model	56
8.4	Alert SMS	67

LIST OF TABLES

Fig. No.	Name of the Tables	Pg. No.
6.1	Normal And Abnormal Conditions	37

NOMENCLATURE

IoT	:	Internet of Things
ECG	:	Electrocardiography
ITU	:	International Telecommunication Union
RFID	:	Radio Frequency Identification
HER	:	Electronic Health Record
AI	:	Artificial Intelligence
RPM	:	Remote Patient Monitoring
CDC	:	Centers for Disease Control
CAGR	:	Compound Annual Growth Rate
DOI	:	Digital Object Identifier
IEEE	:	Institute of Electrical and Electronics Engineers
ICCCIS	:	International Conference on Computing, Communication, and Intelligent Systems
Wi-Fi	:	Wireless Fidelity
MQTT	:	Message Queuing Telemetry Transport
COMST	:	Communications Surveys & Tutorials
LAN	:	Local Area Network
CAD	:	Coronary Artery Disease
PAN	:	Peripheral Artery Disease
LCD	:	Liquid Crystal Display
MHealth	:	Mobile Health
E-Health	:	Electronic Health
ICT	:	Information and Communication Technologies
GSM	:	Global System for Mobile Communication

IDE	:	Integrated Development Environment
SMS	:	Short Message Service
SRAM	:	Static Random Access Memory
EEPROM	:	Electrically Erasable Programmable Read-Only Memory
PWM	:	Pulse Width Modulation
DC	:	Direct Current
SDL	:	Serial Data Line
SCL	:	Serial Clock Line
I2C	:	Inter-Integrated Circuit
RX	:	Receiver
TX	:	Transmitter
SpO2	:	Peripheral Capillary Oxygen Saturation
LED	:	Light Emitting Diode
SPI	:	Serial Peripheral Interface
UART	:	Universal Asynchronous Receiver/Transmitter
VCC	:	Voltage at the Common Collector
GND	:	Ground
OLED	:	Organic Light Emitting Diode
RS	:	Register Select
EN	:	Enable
BP	:	Blood Pressure
HB	:	Hemoglobin
ML	:	Machine Learning
BTS	:	Base Transceiver Station
MSC	:	Mobile Switching Centers

HLR	:	Home Location Register
VLR	:	Visitor Location Register
EDGE	:	Enhanced Data rates for GSM Evolution
HSPA	:	High Speed Packet Access
GPS	:	Global Positioning System
HTTP	:	Hypertext Transfer Protocol

ABSTRACT

The rapid development of the Internet of Things (IoT) has created new opportunities in the healthcare sector by enabling continuous and remote monitoring of patients. Monitoring vital health parameters such as heart rate, oxygen saturation (SpO_2), and body temperature is essential for early detection of medical conditions. Traditional healthcare monitoring systems are usually expensive, limited to hospitals, and require constant supervision by medical staff. Therefore, there is a need for a low-cost and intelligent healthcare monitoring system that can continuously track patient health conditions. This project proposes an IoT and Machine Learning based Smart Health Monitoring System for real-time monitoring of patient health parameters. The system integrates multiple sensors such as MAX30102 for heart rate and SpO_2 measurement, DS18B20 for body temperature monitoring, and DHT11 for measuring room temperature and humidity. These sensors continuously collect physiological and environmental data from the patient.

All the sensors are connected to an Arduino Uno microcontroller, which processes the sensor readings and displays the results on an I2C LCD display for real-time monitoring. The ESP32 Wi-Fi module is used to transmit the collected data to the ThingsSpeak cloud platform for remote monitoring and data storage. Doctors, caregivers, and family members can access the patient's health data anytime through the internet.

In addition, a Machine Learning algorithm is implemented to analyze the collected health data and classify the patient's condition as normal, warning, or critical. This intelligent analysis helps in detecting abnormalities at an early stage and provides timely alerts for preventive healthcare.

The proposed system is low cost, portable, and suitable for home healthcare and remote patient monitoring. By integrating IoT technology with machine learning, the system improves healthcare efficiency and ensures continuous monitoring of patient health conditions outside hospital environments.

Keywords: IoT, Smart Health Monitoring, Machine Learning, Arduino Uno, ESP32, MAX30102, DS18B20, DHT11, Things Speak Cloud, Remote Patient Monitoring, Real-Time Health Monitoring

CHAPTER-1

INTRODUCTION

CHAPTER-1

Introduction

1.1 Motivation of the Research Topic

The rapid advancement of technology has significantly improved the healthcare sector, leading to the development of smart health monitoring systems. Cardiovascular diseases and other chronic illnesses are major causes of death worldwide, creating the need for continuous monitoring of vital health parameters. Important parameters such as Electrocardiogram (ECG), heart rate, blood oxygen saturation (SpO_2), and body temperature help in identifying abnormalities in the human body. Traditional health monitoring systems are mostly hospital-based and require expensive equipment and continuous supervision by medical professionals. These systems are not suitable for continuous monitoring outside hospitals, especially for elderly patients or people living in remote areas.

The development of Internet of Things (IoT) technology enables sensors and medical devices to connect to the internet and transmit patient data in real time. This allows doctors and caregivers to monitor patient health remotely. In addition, Machine Learning (ML) algorithms can analyze the collected health data and detect abnormal patterns or potential health risks. By integrating IoT technology with machine learning techniques, intelligent healthcare systems can be developed to provide early detection of diseases and timely medical alerts.

Therefore, the motivation of this research is to develop a smart, low-cost, and portable health monitoring system that continuously monitors multiple health parameters and provides intelligent analysis to improve patient safety and healthcare services.

1.2 Objective of the Work

The objective of the proposed work is to design and develop an advanced IoT and Machine Learning based Smart Health Monitoring System that enables continuous, real-time monitoring of a patient's vital health parameters. The system aims to measure important physiological signals such as heart rate, blood oxygen saturation (SpO_2), and body temperature, along with environmental conditions like room temperature and humidity. By integrating multiple sensors with

microcontrollers, the objective is to create a reliable and efficient system that can collect accurate health data and display it locally for immediate observation.

Another key objective of this work is to enable remote health monitoring using IoT technology. The system is designed to transmit collected data to a cloud platform, allowing doctors, caregivers, and family members to access the patient's health status from anywhere at any time. This reduces the need for continuous hospital visits and supports home-based healthcare, especially for elderly patients and those with chronic conditions. The objective also includes ensuring data storage with timestamps for future analysis and medical reference.

Furthermore, the project aims to incorporate Machine Learning techniques to enhance the intelligence of the system. Instead of relying only on basic threshold values, the objective is to analyze health data using ML algorithms and classify the patient's condition into categories such as normal, warning, or critical. This helps in early detection of potential health risks and provides timely alerts, improving preventive healthcare and patient safety.

Finally, the overall objective is to develop a low-cost, portable, and user-friendly healthcare solution that can be widely used in real-world applications. The system is intended to be scalable, easy to implement, and capable of improving healthcare accessibility, particularly in remote and rural areas where medical facilities may be limited. By combining IoT connectivity with intelligent analysis, the work aims to bridge the gap between traditional healthcare systems and modern smart monitoring technologies.

1.3 Internet of Things (IoT)

The Internet of Things (IoT) is a technology that enables physical devices to connect to the internet and exchange data with other devices and systems. It allows sensors, machines, and everyday objects to collect, send, and receive information automatically without human intervention. IoT technology plays an important role in modern applications such as smart homes, smart cities, industrial automation, and healthcare systems.

In healthcare, IoT helps in continuous monitoring of patient health parameters using connected medical devices and sensors. These devices collect important physiological data such as heart rate, oxygen saturation, and body temperature and

transmit it to cloud platforms through the internet. Doctors and healthcare professionals can monitor the patient's condition remotely and take necessary actions when abnormal conditions are detected.

The IoT system mainly consists of sensors, microcontrollers, communication networks, cloud storage, and user interfaces. Sensors collect data from the environment or the human body, and microcontrollers process this data. The processed data is then transmitted through communication modules like WiFi or Bluetooth to cloud platforms for storage and analysis.

IoT technology improves healthcare by providing real-time monitoring, remote patient management, early disease detection, and improved medical decision-making. It reduces the need for frequent hospital visits and allows patients to receive continuous care from their homes.

In this project, IoT is used to collect health data from multiple sensors and transmit the data to the cloud using an ESP32 WiFi module. The data is stored on the ThingSpeak platform where it can be accessed remotely for monitoring and analysis. By integrating IoT with machine learning, the system becomes more intelligent and capable of detecting potential health issues at an early stage.

1.3.1 IoT Architecture

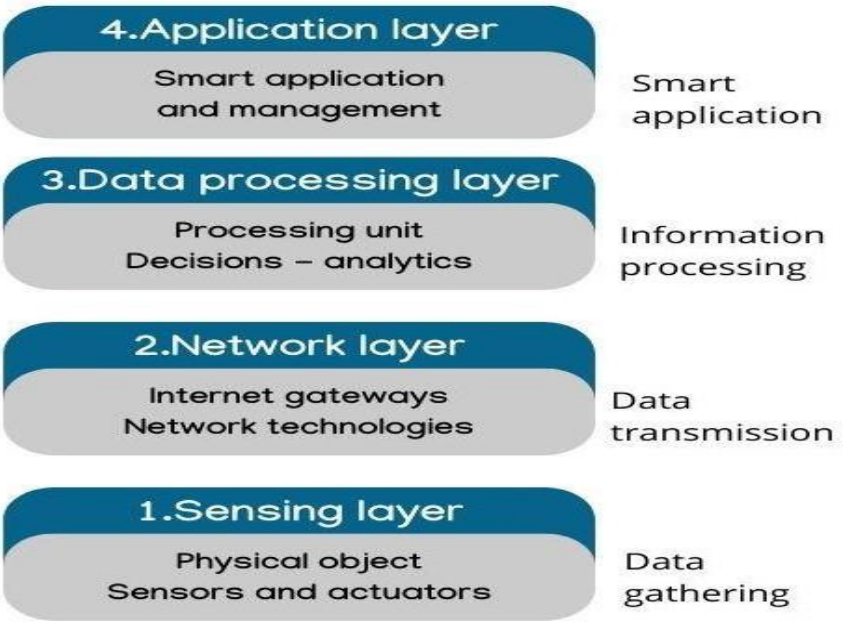


Figure 1.1 IoT Architecture

- **Perception Layer (Sensor Layer)**

The perception layer is the first layer of the IoT architecture. It consists of sensors and devices that collect data from the physical environment. These sensors measure different parameters such as temperature, humidity, heart rate, and oxygen saturation. In healthcare monitoring systems, biomedical sensors are used to collect physiological data from the patient's body. In the proposed system, sensors such as MAX30102, DS18B20, and DHT11 are used to measure heart rate, blood oxygen level, body temperature, and environmental conditions. The collected data is then sent to the microcontroller for further processing.

- **Network Layer (Communication Layer)**

The network layer is responsible for transmitting the collected data from sensors to cloud platforms or servers through communication technologies. It acts as a bridge between the hardware devices and the internet. This layer uses communication technologies such as WiFi, Bluetooth, Zigbee, or cellular networks. In the proposed system, the ESP32 WiFi module is used to send sensor data to the cloud platform. This enables real-time data transmission and remote monitoring of the patient's health parameters.

- **Processing Layer (Middleware Layer)**

The processing layer handles data storage, processing, and analysis. The data received from sensors is processed and stored in databases or cloud platforms. This layer also performs data analysis to extract useful information. Machine learning algorithms can be applied in this layer to analyze patterns and detect abnormalities in the collected data. In the proposed system, the ThingSpeak cloud platform is used to store and visualize the health data for further analysis.

- **Application Layer**

The application layer is the final layer of the IoT architecture that provides services to users based on the processed data. It allows users to access the information through web applications or mobile applications. In healthcare systems, doctors, caregivers, and patients can monitor health parameters remotely through dashboards. This layer also generates alerts when abnormal conditions are detected, helping healthcare professionals take timely action**.**

1.3.2 Applications of IoT

- **Healthcare**

IoT plays an important role in modern healthcare systems by enabling remote patient monitoring and smart medical devices. Sensors and wearable devices continuously monitor health parameters such as heart rate, oxygen saturation, body temperature, and blood pressure. The collected data is transmitted to doctors through cloud platforms for analysis. This helps in early disease detection and reduces the need for frequent hospital visits.

- **Smart Homes**

IoT technology is widely used in smart home systems to automate and control household devices. Smart devices such as lights, thermostats, security cameras, and smart locks can be connected to the internet and controlled remotely using smartphones. This improves convenience, safety, and energy efficiency in homes.

- **Smart Cities**

IoT is used in smart city applications to improve urban infrastructure and services. Sensors are used for traffic monitoring, waste management, smart lighting, and environmental monitoring. These systems help city authorities manage resources efficiently and improve the quality of life for citizens.

- **Agriculture**

In agriculture, IoT is used for smart farming and precision agriculture. Sensors monitor soil moisture, temperature, humidity, and crop conditions. Farmers can use this information to optimize irrigation and fertilizer usage, which increases crop productivity and reduces resource wastage.

- **Industrial Automation**

IoT is also widely used in industries for automation and equipment monitoring. Sensors installed on machines monitor performance and detect faults in real time. This helps industries perform predictive maintenance, reduce downtime, and improve productivity.

1.4 Machine Learning (ML)

Machine Learning (ML) is a branch of artificial intelligence that enables computer systems to learn from data and make predictions or decisions without being explicitly programmed. ML algorithms analyze large amounts of data, identify patterns, and improve their performance over time through experience. It is widely used in many fields such as healthcare, finance, image recognition, speech processing, and smart systems.

In healthcare applications, machine learning plays an important role in analyzing medical data and detecting abnormalities in patient health conditions. ML models can process physiological signals such as heart rate, oxygen saturation (SpO_2), and body temperature to identify unusual patterns that may indicate potential health problems. This helps doctors and healthcare professionals make better decisions and provide timely treatment.

In the proposed smart health monitoring system, machine learning is used to analyze the health data collected from different sensors. The system gathers parameters such as heart rate, blood oxygen level, and body temperature and uses an ML algorithm to classify the patient's health condition. The model analyzes the sensor readings and categorizes the health status into conditions such as Normal, Warning, or Critical.

The ML model can be trained using health datasets so that it can accurately identify abnormal values and patterns. This intelligent analysis helps in early detection of diseases and reduces the risk of serious health complications. It also minimizes the need for manual monitoring by healthcare professionals.

By integrating machine learning with IoT technology, the system becomes more intelligent and capable of performing automated health analysis. This improves the efficiency of the healthcare monitoring system and provides reliable real-time health predictions for better patient care.

1.5 Overview of the Work

1. Data Collection using Sensors

The first step in the proposed system is the collection of health-related data from the patient using various biomedical sensors. These sensors are responsible for

measuring important physiological parameters that indicate the health condition of the patient. The MAX30102 sensor is used to measure heart rate and blood oxygen saturation (SpO_2). The DS18B20 sensor measures the body temperature of the patient. In addition, the DHT11 sensor is used to monitor environmental parameters such as room temperature and humidity. These sensors continuously collect real-time data from the patient and the surrounding environment. Continuous data collection helps in identifying any sudden changes in the patient's health condition.

2. Data Processing using Microcontroller

After collecting the data from sensors, the next step is processing the data using a microcontroller. In this system, the ESP32 microcontroller acts as the central processing unit. The Arduino receives the sensor signals and processes them into readable digital values. It also controls the operation of different components in the system. The microcontroller ensures that the sensor data is properly collected, organized, and prepared for further transmission and analysis.

3. Real-Time Local Display

The processed sensor data is displayed locally using an I2C LCD display. The LCD screen shows the health parameters such as heart rate, blood oxygen level, and body temperature in real time. This allows the patient or user to quickly check the current health status without needing internet access. The real-time display improves user interaction with the system and makes the monitoring process easier.

4. Data Transmission using IoT Technology

To enable remote monitoring, the system uses IoT technology for transmitting the collected data to the internet. The ESP32 WiFi module is used to connect the system to a wireless network. Once connected, the ESP32 sends the sensor data to a cloud platform through the internet. This wireless communication enables continuous and real-time data transmission.

5. Cloud Storage and Data Visualization

The transmitted data is stored on the ThingSpeak cloud platform. The cloud platform organizes and stores the health data along with date and time information. It also provides graphical visualization of the data, which helps users understand the

patient's health trends over time. Doctors and caregivers can access the cloud platform from anywhere to monitor the patient's health condition remotely.

6. Machine Learning Based Data Analysis

In order to make the system more intelligent, Machine Learning algorithms are integrated into the system. The ML model analyzes the collected health data and identifies patterns in the physiological parameters. This analysis helps in detecting abnormal values that may indicate potential health risks.

7. Health Condition Classification

Based on the machine learning analysis, the system classifies the patient's health condition into different categories. These categories include Normal condition, Warning condition, and Critical condition. This classification helps healthcare professionals understand the patient's health status quickly and accurately.

8. Alert and Decision Support System

If the system detects abnormal health parameters, it can generate alerts for doctors, caregivers, or family members. These alerts help in taking immediate medical action and prevent serious health complications. The alert mechanism improves patient safety and supports quick decision making.

9. Continuous Monitoring and Remote Healthcare

The proposed system allows continuous monitoring of patient health conditions in real time. The integration of IoT and Machine Learning makes the system more efficient and intelligent. It enables remote healthcare services, reduces hospital visits, and provides better care for elderly patients and individuals with chronic diseases.

1.6 Why Health Monitoring System?

Health monitoring systems are becoming increasingly important in modern healthcare due to the rising number of chronic diseases and sudden health complications. Continuous monitoring of vital health parameters helps in the early detection of medical conditions and improves patient safety. Traditional healthcare systems mainly rely on hospital-based monitoring, where patients must visit hospitals

frequently for check-ups. This process can be time-consuming, expensive, and inconvenient, especially for elderly patients and those living in remote areas.

A health monitoring system allows continuous observation of important physiological parameters such as heart rate, oxygen saturation (SpO_2), and body temperature. By regularly tracking these parameters, abnormal changes in a patient's health condition can be detected at an early stage. Early detection helps doctors provide timely medical treatment and prevent serious health complications.

Another important reason for using health monitoring systems is the need for remote healthcare services. With the help of IoT technology, patient health data can be transmitted to cloud platforms and accessed by doctors and caregivers from anywhere. This enables remote patient monitoring and reduces the need for frequent hospital visits. It also allows family members to stay informed about the patient's health condition.

Health monitoring systems are particularly useful for elderly individuals, patients with chronic diseases, and people recovering from surgery. Continuous monitoring ensures that any sudden health changes are detected quickly, allowing immediate medical assistance.

Therefore, the development of smart health monitoring systems improves healthcare efficiency, enhances patient safety, and supports better medical decision-making. By integrating technologies such as IoT and Machine Learning, modern health monitoring systems can provide intelligent analysis and real-time health monitoring for better patient care.

1.6 Definition of Health Monitoring System

A Health Monitoring System is a technology-based system designed to continuously measure, record, and analyze important physiological parameters of a patient to assess their health condition. It uses various biomedical sensors and electronic devices to collect health-related data such as heart rate, body temperature, blood oxygen saturation (SpO_2), and other vital signs. The collected data helps in monitoring the patient's health status in real time and detecting any abnormal changes.

In modern healthcare systems, health monitoring systems are often integrated with technologies such as Internet of Things (IoT), cloud computing, and machine learning to enable remote monitoring and intelligent data analysis. These systems allow doctors, caregivers, and family members to access patient health information from anywhere through the internet. By providing continuous monitoring and early detection of health issues, health monitoring systems help improve patient safety, reduce hospital visits, and support better medical decision-making.

1.7 History of Health Monitoring System

The concept of health monitoring systems has evolved significantly with the advancement of medical technology and electronics. In the early days, patient health monitoring was performed manually by doctors and nurses using basic medical instruments such as thermometers, stethoscopes, and blood pressure monitors. These methods required patients to visit hospitals or clinics regularly, and continuous monitoring was not possible. As a result, detecting sudden changes in a patient's health condition was difficult.

With the development of electronic medical devices in the late 20th century, hospitals began using automated monitoring systems to measure vital parameters such as heart rate, ECG signals, blood pressure, and body temperature. These systems allowed healthcare professionals to continuously observe patients in intensive care units (ICUs). However, these monitoring systems were large, expensive, and limited to hospital environments.

In recent years, the advancement of wireless communication, embedded systems, and the Internet of Things (IoT) has transformed healthcare monitoring systems. Modern health monitoring systems use wearable sensors, microcontrollers, and cloud platforms to continuously monitor patient health parameters in real time. These systems allow remote monitoring of patients from anywhere through the internet.

Furthermore, the integration of Machine Learning and Artificial Intelligence has improved the intelligence of health monitoring systems. As a result, modern health monitoring systems are becoming more efficient, portable, and suitable for home healthcare and remote patient monitoring applications.

CHAPTER-2

LITERATURE SURVEY

CHAPTER-2
Literature Survey

1. [36] Gupta, A., Singh, R., & Kumar, P. (2023). IoT-Based Smart Healthcare Monitoring System Using Wearable Sensors. In 2023 IEEE International Conference on Smart Computing and Electronic Enterprise (ICSCEE). IEEE.

Above reference is a research paper titled "IoT-Based Smart Healthcare Monitoring System Using Wearable Sensors" authored by A. Gupta, R. Singh, and P. Kumar. It was presented at the 2023 IEEE International Conference on Smart Computing and Electronic Enterprise. The paper focuses on developing an IoT-based healthcare monitoring system that uses wearable sensors for continuous monitoring of patient health parameters.

The proposed system integrates sensors such as heart rate sensors, temperature sensors, and oxygen saturation sensors to monitor vital health parameters of patients. These sensors collect real-time physiological data and transmit it to healthcare monitoring systems using wireless communication technologies.

Furthermore, the study highlights that IoT-based wearable healthcare systems enable remote patient monitoring and early detection of abnormal health conditions. This approach improves healthcare accessibility and allows doctors to monitor patients from remote locations.

2. [37] Khan, M., Ahmed, S., & Ali, T. (2023). Real-Time IoT-Based Patient Health Monitoring System Using Cloud Computing. In 2023 IEEE International Conference on Internet of Things and Intelligence System (IoTaIS). IEEE.

Above reference is an article titled "Real-Time IoT-Based Patient Health Monitoring System Using Cloud Computing" authored by M. Khan, S. Ahmed, and T. Ali. It was presented at the *IEEE IoTaIS Conference* in 2023.

The paper discusses the development of an IoT-based healthcare monitoring system designed to collect physiological data from patients using biomedical sensors. The collected data is transmitted to cloud platforms where it can be stored and analyzed.

The authors explain that cloud-based healthcare monitoring systems enable doctors and healthcare providers to monitor patient health conditions in real time. This system also provides alerts when abnormal health parameters are detected, improving patient safety.

3. [38] Patel, D., & Shah, H. (2024). Smart Health Monitoring System Using IoT and Machine Learning. In 2024 IEEE International Conference on Artificial Intelligence and Smart Systems (ICAISS). IEEE.

Above reference is a research paper titled **"**Smart Health Monitoring System Using IoT and Machine Learning" authored by D. Patel and H. Shah. It was presented at the *2024 IEEE International Conference on Artificial Intelligence and Smart Systems.*

The paper focuses on integrating IoT technology with machine learning algorithms to develop an intelligent healthcare monitoring system. The system collects patient health parameters using IoT sensors and analyzes the data using machine learning techniques.

Furthermore, the study explains that machine learning algorithms can identify abnormal patterns in health data and predict potential health risks. This approach improves disease detection and supports intelligent healthcare monitoring systems.

4. [39] Chatterjee, S., Roy, A., & Mukherjee, S. (2024). AI-Enabled IoT Framework for Remote Healthcare Monitoring. IEEE Access, 12, 45520-45535. DOI: 10.1109/ACCESS.2024.3364512.

Above reference is an article titled "AI-Enabled IoT Framework for Remote Healthcare Monitoring" authored by S. Chatterjee, A. Roy, and S. Mukherjee. It was published in the *IEEE Access* journal in 2024.

The article discusses the development of an intelligent IoT framework that integrates artificial intelligence techniques for healthcare monitoring. The system collects patient health data using sensors and transmits the data to cloud platforms for analysis.

The authors highlight that AI-based IoT healthcare systems enable early detection of diseases by analyzing large volumes of patient health data. This technology helps healthcare professionals provide accurate diagnosis and improve healthcare services.

5. [40] Ramesh, S., & Prakash, V. (2025). Intelligent IoT-Based Healthcare Monitoring System Using Edge Computing. In 2025 IEEE International Conference on Smart Internet of Things (SmartIoT). IEEE.

Above reference is a research paper titled “Intelligent IoT-Based Healthcare Monitoring System Using Edge Computing” authored by S. Ramesh and V. Prakash. It was presented at the *2025 IEEE International Conference on Smart Internet of Things.*

The paper discusses an advanced healthcare monitoring system that integrates IoT devices with edge computing technologies. The system collects health data using biomedical sensors and processes the data locally using edge computing devices.

This approach reduces latency and enables faster detection of abnormal health conditions. The study highlights that edge computing combined with IoT can significantly improve the efficiency and reliability of healthcare monitoring systems.

CHAPTER-3

PATIENT HEALTH MONITORING SYSTEM IN IOT WITH ML

CHAPTER-3

Patient Health Monitoring System in IoT with ML

The aim of this work is to develop a low-cost, portable healthcare monitoring system using IoT technology to address the challenges of accessing specialized medical care in remote areas. The scope includes designing user-friendly interfaces, exploring applications in disaster response and telemedicine, and ensuring scalability, interoperability, and data security.

3.1 Aim of Patient Health Monitoring System :

The primary goal of this work is to develop a smart patient health monitoring system in such a way that one can get all the necessary and detailed information of the disease. The proposed system measures the body temperature, pulse rate and ECG data [1]. Health is always a major concern in every growth human is advancing in terms of technology. Like the recent corona virus attack that has ruined the economy of China to an extent is an example how health care has become of major importance. In such areas where the epidemic is spread, it is always a better idea to monitor these patients using remote health monitoring technology [2]. Remote Patient Monitoring arrangement empowers observation of patients outside of customary clinical settings, which expands access to human services offices at bring down expenses. The fundamental element of people's needs is health [3]. Humans face a haul of surprising death and plenty of diseases because of varied diseases that are a result of lack of treatment to the patients at right time.

The sensors will be either worn or be embedded into the body of the patients, to unendingly monitor their health. the knowledge collected in such a fashion will behold on, analyzed, and well-mined to try and do the first prediction of diseases [4]. The concept of Internet of things is recent and is defined as the integration of all devices that connect to the network, which can be managed from the web and in turn provide information in real time, to allow interaction with people they use it [5]. Another concept of IoT "is the general idea of things, especially everyday objects, which are readable, recognizable, locatable, addressable and controllable via the Internet- either through RFID, wireless LAN, wide area network, or by other means "[6]. IoT The term itself was first mentioned by Kevin Ashton in 1998 and aims at the exchange of information [7]. On the other hand [8], the Internet of things can be seen from three paradigms, which are Internet-oriented middleware, things sensors

oriented and knowledge-oriented semantics. Therefore, it is appropriate, such delimitation because the interdisciplinary nature of the subject. However, the usefulness of the IoT is reflected when crossing between the three paradigms in the development of applications [9].

3.2 Scope Patient Health Monitoring System in IoT:

This work will help in monitoring the patient's health which will be helpful for doctors and patients both. It will help in reducing and early prediction of disease. The core objective of this work is the design and implementation of a smart patient health tracking system that uses Sensors to track patient health and uses internet to inform their loved ones in case of any issues [10]. The objective of developing monitoring systems is to reduce health care costs by reducing physician office visits, hospitalizations, and diagnostic testing procedure Each of the bodies utilizes temperature and pulse acknowledging to peruse understanding wellbeing.

The sensors are linked to a microcontroller to track the status which is thus interfaced to an LCD screen and additionally remote association with have the capacity to exchange alarms. If framework finds any sudden changes in understanding heartbeat or body temperature, the framework consequently alarms the client about the patient's status over IoT and furthermore indicates subtle elements of pulse and temperature of patient live in the web. In this manner IoT set up tolerant wellbeing following framework viably utilizes web to screen quiet wellbeing measurements and spare persists time.

The increased use of mobile technologies and smart devices in health has caused great impact on the world. Health experts are increasingly taking advantage of the benefits these technologies bring, this generating a significant improvement in health care in clinical settings and out of them. Likewise, countless ordinary users are being served from the advantages of the MHealth applications and E-Health to improve, help and assist their health. Applications that have had a major refuge for these users, so intuitive environment [11]. The Internet of things is increasingly allowing to integrate devices capable of connecting to the Internet and provide information on the state of health of patients and provide information in real time to doctors who assist. The potential of this patient health tracking system extends far beyond simply informing loved ones. By continuously monitoring vital signs and other health data, the system can play a crucial role in early disease prediction and

prevention. Deviations from normal baselines in temperature, heart rate, or other readings can serve as early warning signs, prompting further investigation and potentially leading to earlier diagnoses and interventions. This proactive approach to healthcare can significantly improve patient outcomes and quality of life.

Furthermore, the remote monitoring capabilities of this system can empower patients to take a more active role in managing their own health. By having access to real-time data and insights, patients can make informed decisions about their daily routines and lifestyle choices. This can be particularly beneficial for individuals with chronic conditions, allowing them to self-monitor and adjust their behavior to maintain optimal health. Looking ahead, the integration of this patient health tracking system with other healthcare technologies holds immense promise. By seamlessly connecting with electronic medical records and telehealth platforms, the system can create a comprehensive view of a patient's health. This interconnected ecosystem of data can empower healthcare providers to deliver more personalized and effective care, ultimately leading to a future of preventative, proactive, and patient-centered

CHAPTER-4

PROPOSED SYSTEM DESIGN

CHAPTER-4

Block Diagram And Circuit Diagram

4.1 BLOCK DIAGRAM

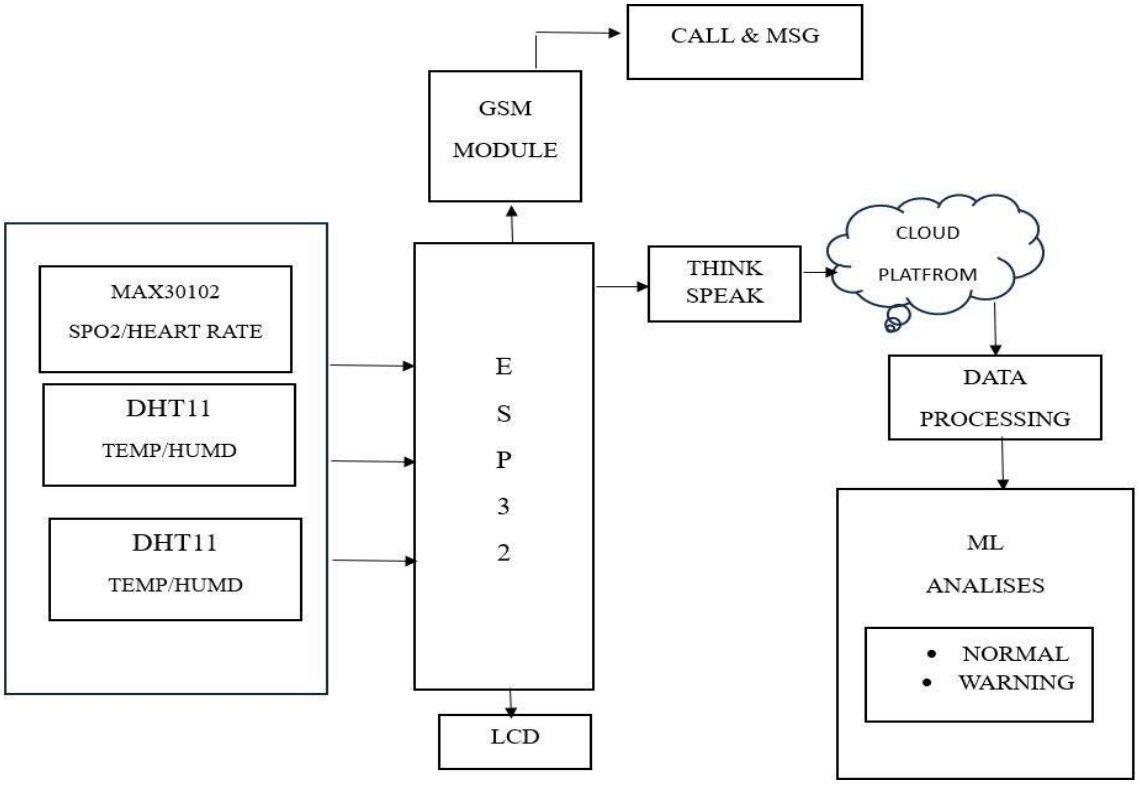


Figure 4.1: Block Diagram of Health Monitoring System

The block diagram represents the overall architecture and working principle of the proposed IoT and Machine Learning based smart health monitoring system. The system consists of various hardware modules including biomedical sensors, a microcontroller unit, display unit, communication module, and cloud platform for remote monitoring. These modules work together to continuously monitor the patient's vital parameters and analyze the health condition in real time.

The system uses multiple sensors such as the MAX30102 sensor, DS18B20 temperature sensor, and DHT11 sensor to measure important physiological and environmental parameters. The MAX30102 sensor measures the patient's heart rate and blood oxygen saturation (SpO_2) levels. The DS18B20 sensor is used to monitor the body temperature of the patient, while the DHT11 sensor measures room temperature and humidity surrounding the patient. These sensors continuously collect the data and send it to the microcontroller for processing.

4.2 FLOW CHART

The given flowchart represents the working process of an IoT and Machine Learning based Smart Health Monitoring System. It begins with the system startup, where all components such as Wi Fi, sensors, LCD display, and GSM module are initialized to ensure proper functioning. Once the system is ready, it proceeds to the mode selection stage, where the user can choose between Auto Mode and Manual Mode depending on how the data is to be provided. In Auto Mode, the system automatically fetches patient health data such as heart rate (HR), SpO_2, body temperature, and room temperature from the ThingSpeak cloud using an API. In contrast, in Manual Mode, the user manually enters these health parameters into the system. Regardless of the mode selected, the collected data is then sent to the Machine Learning (ML) model for analysis.The ML model processes the input data and predicts the patient's health condition, classifying it as either normal or critical. After prediction, the system checks whether the condition is critical. If the condition is normal, the system displays a "NORMAL" status on the user interface, generates an AI-based explanation indicating a healthy condition, and updates the data on the ThingSpeak dashboard for monitoring and record-keeping.If the condition is identified as critical, the system displays a "CRITICAL" alert on the user interface and generates an AI-based explanation describing the abnormal condition. Additionally, it takes immediate action by sending an SMS alert through the GSM module and making a call to notify caregivers or medical professionals. This ensures timely intervention in emergency situations. Finally, after completing the process in either case, the system continues monitoring the patient's health in a loop, enabling continuous real-time tracking and analysis.

It receives the input signals from all sensors, processes the data, and converts the raw sensor values into meaningful health parameters. The processed data is then displayed locally using a 16×2 I2C LCD display, allowing the patient or caregiver to view the real-time health information. For remote monitoring and Internet connectivity, an ESP32 WiFi module is integrated with the Arduino. The ESP32 transmits the collected health data to the ThingSpeak cloud platform, where the data is stored along with date and time information. This enables doctors, caregivers, and family members to access the patient's health information from anywhere through the internet.To enhance the intelligence of the system, a Machine Learning algorithm is used to analyze the collected health data. The algorithm evaluates the sensor readings and classifies the patient's health condition into different categories such as Normal, Moderate Risk, or Critical condition. This intelligent analysis helps in early detection of abnormal health conditions and improves preventive healthcare

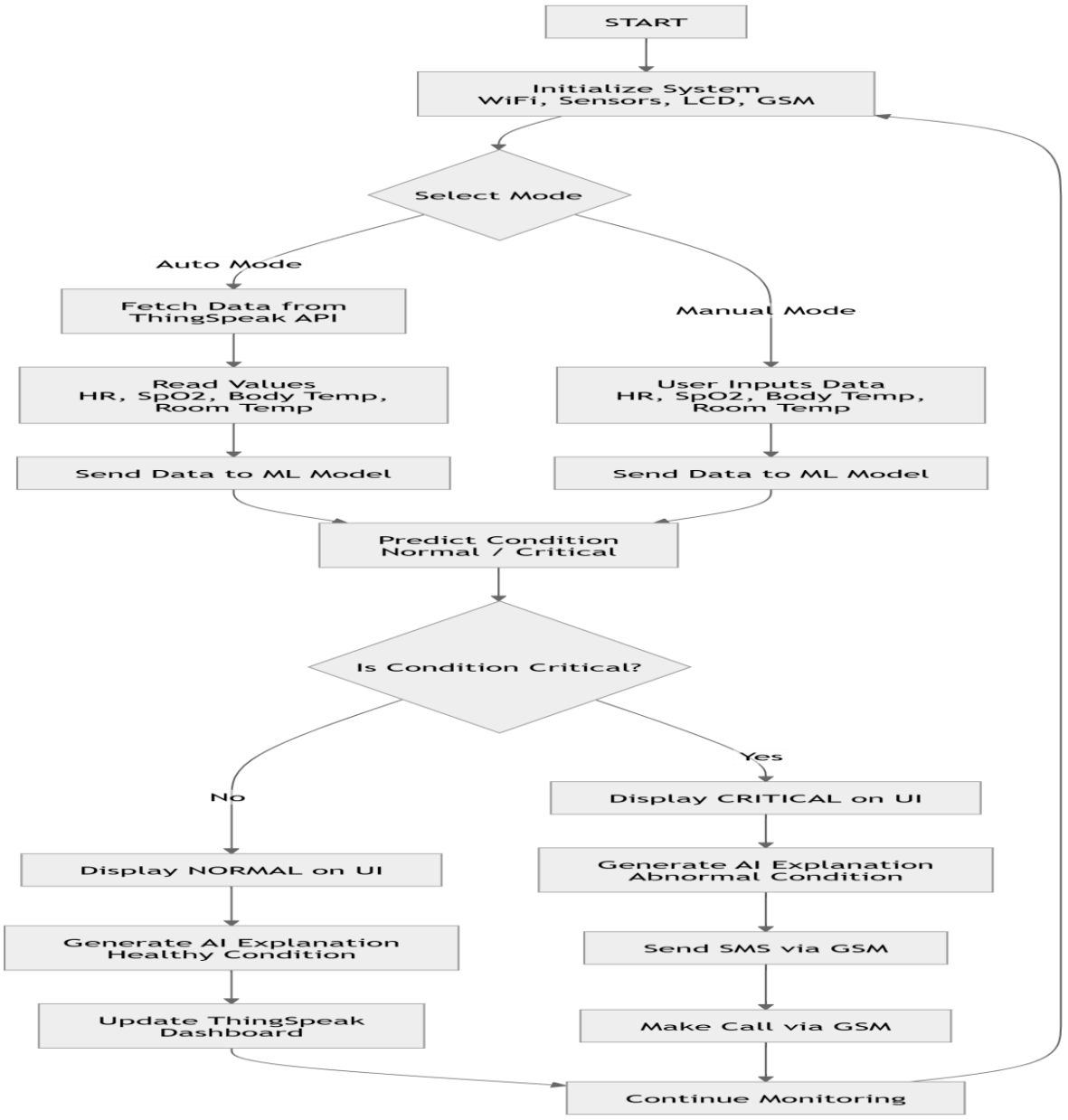


Figure 4.2: Flow Chart of Health Monitoring System

4.3 CIRCUIT DIAGRAM

The circuit represents an IoT-based health monitoring system built around the ESP32 microcontroller, which acts as the central processing unit. It receives input data from multiple sensors, processes the information, and controls output devices like the LCD display and GSM module. The ESP32 uses both digital and communication pins such as GPIO21 and GPIO22 for I2C communication, GPIO4 and GPIO5 for sensor data, and UART pins (GPIO16 and GPIO17) for GSM communication. The power supply section provides a regulated 5V DC output to the system. This voltage is distributed to components like the ESP32, DHT22 sensor, LCD display, and MAX30102 module. A common ground is maintained across all components to ensure proper

operation and stable signal reference. The ESP32 internally provides 3.3V, which is used by sensors like the MAX30102 and DS18B20 that require lower operating voltage. The MAX30102 sensor is used to measure heart rate and blood oxygen (SpO_2) levels. It communicates with the ESP32 using the I2C protocol, where SDA is connected to GPIO21 and SCL to GPIO22. Since I2C is a shared communication bus, multiple devices like the LCD can use the same two lines without conflict, as each device has a unique address.

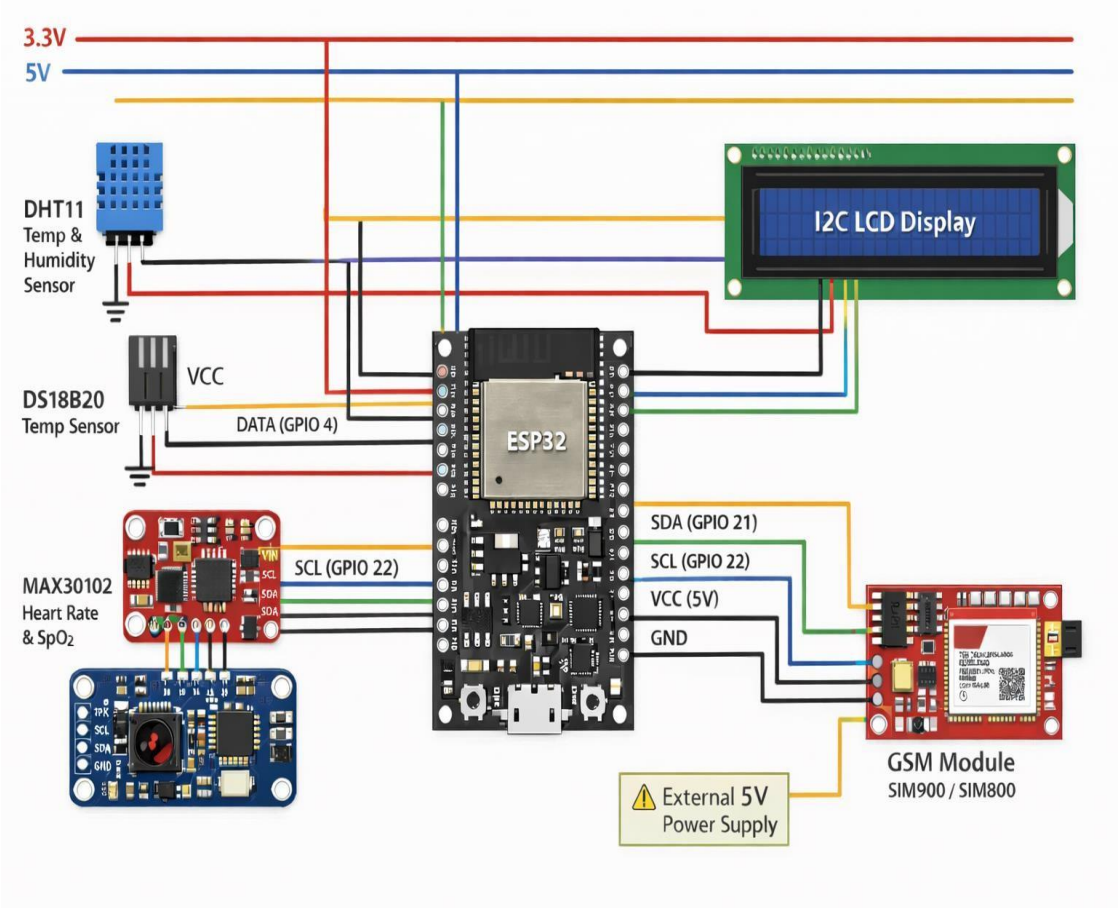


Figure 4.3: Circuit Diagram of Health Monitoring System

The DHT22 sensor measures ambient temperature and humidity. It is connected to one of the ESP32's digital pins (GPIO4) and operates on a single-wire communication protocol. The sensor is powered using the 5V supply and shares a common ground with the ESP32. The DS18B20 sensor is used for precise temperature measurement, often for body temperature monitoring. It communicates with the ESP32 through a one-wire protocol connected to GPIO5. A 4.7kΩ pull-up resistor is placed between the data line and VCC to ensure reliable communication. The I2C LCD (16x2 display) is used to display real-time sensor readings such as heart rate, SpO_2 levels, and temperature values. It requires only four connections: VCC (5V), GND, SDA (GPIO21), and SCL (GPIO22). It shares the same I2C lines with the MAX30102 sensor, making the circuit simpler and

reducing wiring complexity.The SIM800L GSM module is used for wireless communication, mainly to send SMS alerts in case of abnormal health conditions. It communicates with the ESP32 using UART protocol, where the module's TX pin is connected to ESP32 RX (GPIO16) and RX pin to TX (GPIO17). It requires a stable power supply and a common ground, along with an antenna for network connectivity. Overall, the system continuously monitors vital parameters using sensors, processes the data through the ESP32, displays the readings on the LCD, and sends alerts via the GSM module when necessary.

CHAPTER-5

WORK IMPLEMENTATION

CHAPTER-5

WORK IMPLEMENTATION

5.1 Existing System:

Existing System The existing system is a wearable IoT-based Tele-ECG monitoring system designed to measure and transmit ECG signals and heart rate data for remote healthcare monitoring. The system uses textile electrodes integrated into wearable clothing to capture ECG signals. The ECG signal is processed and transmitted through wireless communication modules such as Bluetooth to smartphones or servers. The data can then be accessed remotely by healthcare professionals through web interfaces or mobile applications. The system also includes an SMS-based alert mechanism that sends notifications to physicians when abnormal heart rate values are detected or when the patient presses a help button. Although the system provides real-time ECG monitoring and remote data transmission, it mainly focuses on ECG signals and heart rate measurements.

5.2 Proposed System:

Proposed System The proposed system is an IoT and Machine Learning based Smart Health Monitoring System that continuously monitors multiple physiological and environmental parameters of a patient. The system integrates the following sensors: • MAX30102 Sensor – Measures Heart Rate and Blood Oxygen Saturation (SpO_2) • DS18B20 Sensor – Measures Body Temperature • DHT11 Sensor – Measures Room Temperature and Humidity All sensors are connected to an Arduino Uno microcontroller, which collects and processes the sensor data. The measured values are displayed locally on an I2C LCD display for real-time monitoring. For remote monitoring and cloud connectivity, an ESP32 module is integrated with the Arduino Uno. The ESP32 uploads the collected data to the ThingSpeak cloud platform, where it is stored along with date and time information. To enhance the intelligence of the system, a Machine Learning algorithm is implemented to analyze the collected health data. The algorithm evaluates the sensor values and classifies the patient's health condition into categories such as: • Normal Condition • Moderate Risk Condition • Critical Health Condition The ML model is trained using health parameter datasets to provide accurate predictions and early warnings. This system allows doctors, caregivers, and family members to monitor the patient's health remotely through cloud interface

5.3 Work Implementation:

The work implementation of the proposed IoT and Machine Learning based Smart Health Monitoring System involves the development of a system that can continuously monitor important health parameters of a patient in real time. The system integrates biomedical sensors, a microcontroller, communication modules, and cloud services to collect, process, and transmit health data. The main aim of the implementation is to design a reliable and cost-effective system capable of monitoring parameters such as heart rate, blood oxygen saturation (SpO_2), body temperature, and environmental conditions. The collected data is displayed locally on an LCD display and also transmitted to a cloud platform so that doctors and caregivers can monitor the patient's health condition remotely.

The hardware implementation of the system consists of several sensors and electronic components connected to the microcontroller. The MAX30102 sensor is used to measure the patient's heart rate and blood oxygen saturation levels. This sensor works based on the photoplethysmography technique, which detects changes in blood flow by using light absorption. The DS18B20 digital temperature sensor is used to measure the body temperature of the patient with high accuracy. In addition to monitoring physiological parameters, the system also measures environmental conditions using the DHT11 sensor, which detects room temperature and humidity. These sensors continuously collect data and send it to the microcontroller for processing.

The microcontroller acts as the central processing unit of the system. It receives signals from the sensors, processes the data, and converts it into meaningful values. A 16×2 LCD display is connected to the microcontroller to display the real-time readings of heart rate, oxygen saturation, body temperature, and environmental parameters. This display allows the patient or caregiver to easily observe the current health condition.

For remote monitoring, a wireless communication module is integrated into the system. The communication module sends the collected data to a cloud platform through an internet connection. The cloud platform stores the data along with date and time information, allowing doctors and caregivers to monitor the patient's health condition from anywhere. In some cases, a GSM module with a SIM card can also be used to send alert messages when abnormal health conditions are detected.

The software part of the system is developed using a microcontroller programming environment where the program is written in Embedded C language. The software continuously reads the sensor values, processes the data, and updates the display and cloud platform. The

program also manages communication between the sensors, microcontroller, and communication module. Sensor libraries and communication protocols are used to simplify the programming and ensure efficient data transfer.

Machine Learning techniques can also be applied to the collected health data to analyze patterns and predict the patient's health condition. The algorithm processes the sensor data and classifies the patient's condition into categories such as normal, warning, or critical. This intelligent analysis helps in the early detection of potential health problems and improves preventive healthcare.

After completing the hardware and software integration, the system is tested to ensure proper functioning. Each sensor is verified to check whether it produces accurate readings. The communication module is also tested to confirm that the data is successfully transmitted to the cloud platform. The system is observed under different conditions to ensure continuous monitoring without interruption. The testing results show that the system is capable of monitoring multiple health parameters and providing reliable real-time health information.

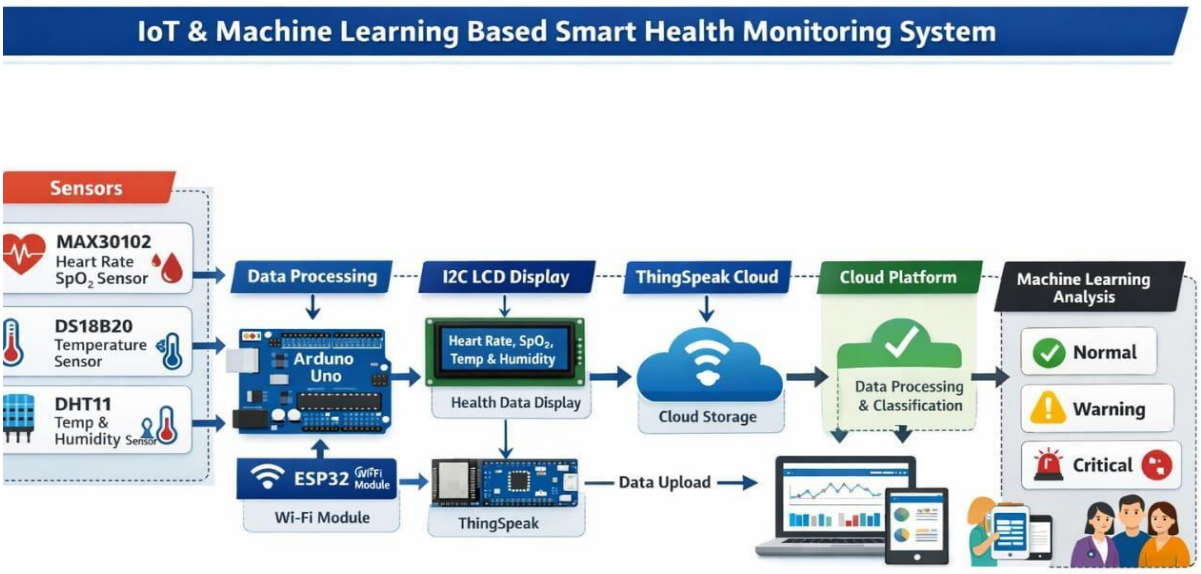


Figure 5.1: Work Implementation Flow

5.3.1 Hardware Requirements

The hardware components required for the proposed smart health monitoring system are listed below:

1. **Microcontroller (ESP32)** – Used as the main processing unit to collect and process sensor data.

2. **MAX30102 Sensor** – Used to measure heart rate and blood oxygen saturation (SpO_2).
3. **DS18B20 Temperature Sensor** – Used to measure the body temperature of the patient.
4. **DHT11 Sensor** – Used to measure room temperature and humidity.
5. **16×2 LCD Display (I2C)** – Used to display real-time health parameters.
6. **GSM Module (SIM900 / SIM800)** – Used to send SMS alerts through a SIM card when abnormal conditions are detected.
7. **Wi-Fi Module (ESP32)** – Used to upload sensor data to the cloud platform for remote monitoring.
8. **Resistors** – Used for proper circuit operation and sensor interfacing.
9. **Jumper Wires** – Used to connect different electronic components.
10. **General Circuit Board (GCB)** – Used to assemble the circuit connections.
11. **Power Supply (12V)** – Used to provide electrical power to all components.

5.3.2 Software Requirements

The software requirements for implementing the proposed smart health monitoring system are listed below:

1. **Arduino IDE** – Used to write, compile, and upload the program to the microcontroller.
2. **Embedded C Programming Language** – Used for developing the control program for sensors and communication modules.
3. **ThingSpeak Cloud Platform** – Used for storing, visualizing, and monitoring sensor data remotely through the internet.
4. **ESP32 Wi-Fi Libraries** – Used to establish internet connectivity and transmit data to the cloud server.
5. **Serial Communication Protocol** – Used for communication between microcontroller and communication modules.
6. **Machine Learning Tools (Python / Cloud ML)** – Used to analyze health data and classify patient conditions such as normal, warning, or critical.

CHAPTER-6

HARDWARE REQUIREMENTS AND PIN SPECIFICATIONS

CHAPTER-6

Hardware Requirements and Pin Specifications

6.1 GPRS Module

This is a SIM900A GSM/GPRS module, a widely used communication module in IoT and embedded systems. It allows your microcontroller (like Arduino or ESP32) to communicate over the cellular network using a SIM card. Main Features, Works on GSM network (2G) ,Send & receive SMS messages , Make & receive calls , Access GPRS (mobile data),Controlled using AT commands via serial communication

A. Pin connections:
   - **VCC** – Connected to External 5V/12V Power Supply
   - **GND** – Connected to Ground
   - **TXD** – Connected to ESP32 RX
   - **RXD** – Connected to ESP32 TX



Figure 6.1:GPRS Module

A GPRS module is a compact communication device that enables embedded systems and IoT projects to connect to the internet using cellular networks instead of Wi-Fi. It operates on General Packet Radio Service (GPRS), which is a packet-based data service available on 2G and 3G networks. Modules like the SIM900A contain an internal modem, SIM card interface, and antenna support, allowing them to transmit voice, SMS, and data. These modules communicate with microcontrollers such as ESP32 or Arduino using AT commands over UART (serial communication), making them easy to control through simple code.

One of the key advantages of a GPRS module is its ability to work in remote and rural areas where Wi-Fi or broadband connectivity is not available. It can continuously send real-time data, such as temperature, heart rate, or GPS location, to cloud platforms like ThingSpeak or a private server. This makes it highly useful in IoT-based health monitoring systems, where patient data can be sent instantly to doctors, and alerts can be triggered via SMS in emergency situations. It is also widely used in vehicle tracking systems, industrial monitoring, and smart agriculture for remote supervision.

From a hardware perspective, the GPRS module includes pins for power supply, ground, transmit (TX), receive (RX), and sometimes control pins like reset and power key. It requires a stable external power supply (usually capable of delivering up to 2A current) because GSM communication involves sudden power bursts. The module also uses an external antenna to maintain a strong network signal. Once powered and connected, it registers with the cellular network, which is usually indicated by an LED blinking pattern.

6.2 ESP32

The ESP32 is a powerful, low-cost microcontroller developed by Espressif Systems, widely used in IoT (Internet of Things) applications. It comes with built-in Wi-Fi and Bluetooth, which allows devices to connect to the internet and communicate wirelessly without needing additional modules. This makes it highly suitable for smart systems like health monitoring home automation, and industrial control.



Figure 6.2:ESP32

Pin connections:

- **3.3V** – Connected to Sensor VCC pins
- **GND** – Connected to All sensor GND pins
- **GPIO21 (SDA)** – Connected to I2C SDA (Sensors/LCD)
- **GPIO22 (SCL)** – Connected to I2C SCL (Sensors/LCD)
- **GPIO4** – Connected to DHT Sensor DATA
- **GPIO5** – Connected to DS18B20 DATA
- **GPIO16 (RX)** – Connected to GSM TX
- **GPIO17 (TX)** – Connected to GSM RX

One of the main advantages of the ESP32 is its high performance and versatility. It features a dual-core processor, multiple GPIO pins, and supports various communication protocols such as UART, I2C, SPI, and PWM. Because of this, it can easily interface with different sensors like temperature sensors, pulse sensors, and displays. It also supports analog and digital inputs, enabling it to collect real-time data from the environment.

The ESP32 is commonly programmed using the Arduino IDE or ESP-IDF, making it beginner-friendly as well as powerful for advanced users. In IoT projects, the ESP32 collects data from connected sensors, processes it, and sends it to cloud platforms like ThingSpeak using Wi-Fi. This allows users to monitor data remotely from anywhere in the world.

Another important feature of the ESP32 is its low power consumption, which makes it ideal for battery-operated devices. It includes different power-saving modes such as deep sleep, which helps extend battery life in long-term monitoring systems. Overall, the ESP32 is a compact, efficient, and flexible microcontroller that plays a crucial role in building modern smart and connected systems.

6.3 MAX30102 (Pulse Rate & SpO_2 Sensor)

The MAX30102 is an advanced sensor used to measure heart rate and blood oxygen (SpO_2) levels. It works using infrared and red light to detect blood flow changes in the fingertip. This sensor communicates with the ESP32 using the I2C protocol, which requires SDA and SCL pins. It is widely used in health collected from this sensor helps in continuous monitoring of a patient's health



Figure 6.3:MAX30102

Pin connections:

- **VCC** -Connected to 3.3V
- **GND** – Connected to Ground
- **SDA** – Connected to GPIO21
- **SCL** – Connected to GPIO22

6.4 DS18B20 (Body Temperature Sensor)

The DS18B20 is a digital temperature sensor used for measuring temperature with good accuracy. It uses the OneWire communication protocol**,** meaning it requires only one data pin to communicate with the ESP32. A pull-up resistor is needed for proper operation. This sensor is especially useful in medical applications where precise temperature monitoring is required.

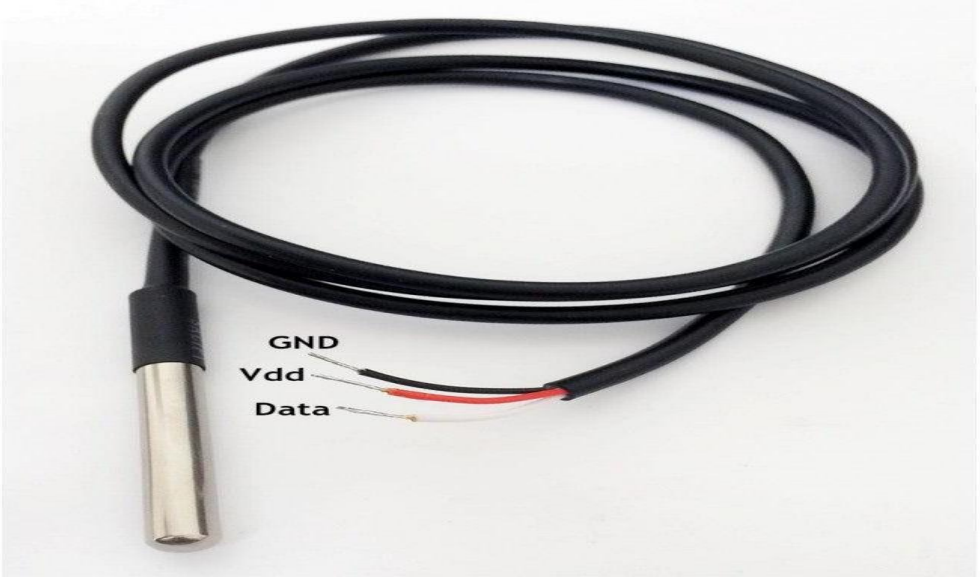


Figure 6.4:DS18B20

Pin connections:

- **VCC** – Connected to 3.3V
- **GND** – Connected to Ground
- **DATA** – Connected to GPIO5
- **Pull-up Resistor** – 4.7kΩ between VCC and DATA

6.4 DHT11 (Temperature & Humidity Sensor)

The DHT11 or DHT22 sensor is used to measure ambient temperature and humidity. It sends digital signals to the ESP32 through a single data pin. This sensor is simple, low-cost, and suitable for basic environmental monitoring. In your project, it helps track surrounding conditions, which can be important for patient comfort and overall health analysis



Figure 6.5:DHT11

Pin connections:

- **VCC** – Connected to 3.3V
- **GND** – Connected to Ground
- **DATA** – Connected to GPIO4

6.5LCD Display (16x2 I2C)

The 16x2 LCD display is one of the most commonly used output devices in embedded systems. It can display 16 characters per row and 2 rows, making it suitable for showing short messages, sensor values, and system status. In your project, it is used to display important health parameters like heart rate, SpO_2 levels, and temperature, allowing users to monitor data directly without needing a mobile or cloud access.



Figure 6.6:LCD Display

This LCD works based on the HD44780 controller, which manages how characters are displayed on the screen. Normally, this type of LCD requires many pins (around 16), but by using an I2C module (backpack), the number of required connections is reduced to just 4 pins (VCC, GND, SDA, SCL). This makes wiring simple and efficient, especially when using microcontrollers like ESP32 that support I2C communication.

The LCD display uses a liquid crystal layer that does not emit light by itself, so it includes a backlight (usually blue or green) for visibility. It also has a small potentiometer (blue knob) on

the I2C module, which is used to adjust the contrast of the display. If the screen shows blank or unclear text, adjusting this knob will make the text visible.

Pin connections

- **VCC** – Connected to **5V**
- **GND** – Connected to **Ground**
- **SDA** – Connected to **GPIO21 (ESP32 SDA)**
- **SCL** – Connected to **GPIO22 (ESP32 SCL)**

Table no: 6.1 Normal and Abnormal Conditions

Sensor	Parameter	Measurement Range	Normal Range	Critical Condition
MAX30102	Heart Rate	30 – 220 bpm	60 – 100 bpm	< 50 or > 110 bpm
MAX30102	SpO_2	0 – 100 %	95 – 100 %	< 90 %
DS18B20	Body Temperature	-55°C to 125°C	36 – 37.5°C	> 38.5°C or < 35°C
DHT11	Room Temperature	0°C – 50°C	20 – 30°C	< 15°C or > 35°C
DHT11	Humidity	20% – 90% RH	40% – 70% RH	< 30% or > 80% RH

CHAPTER-7

SOFTWARE REQUIREMENTS

CHAPTER-7
SOFTWARE REQUIREMENTS

7.1 ARDUINO IDE:

The Arduino Software IDE, also known as the Arduino Integrated Development Environment, is your one-stop shop for creating programs sketches that control Arduino hardware. This software provides a user-friendly interface with all the essential tools you need to bring your ideas to life.

7.1.1 Dissecting the Interface:

- **Text Editor:** You'll write your code sketches using this built-in text editor. It offers standard functionalities like cut, copy, paste, search, and replace to streamline your coding process.
- **Message Area:** Consider this your personal feedback assistant. It keeps you informed during the saving, exporting, and compilation process. Additionally, it displays any errors you encounter in your code, helping you identify and fix them efficiently.
- **Text Console:** This window acts as a communication channel, displaying text output generated by the Arduino software. It provides valuable information, including detailed error messages and other notifications.
- **Configuration Panel:** The bottom right corner of the IDE keeps you in control. It displays the currently selected Arduino board and the corresponding serial port for communication.
- **Toolbar for Efficiency:** A set of handy buttons adorns the toolbar, allowing you to perform essential actions with ease. You can verify and upload your sketches to the Arduino board, manage your sketches create, open, save, and access the serial monitor for communication purposes.

7.1.2 A Note on File Extensions:

Earlier versions of the Arduino IDE prior to version 1.0 used the `.pde` extension for sketch files. While version 1.0 can still open these files, you'll be prompted to save them with the new `.ino` extension upon saving.

In essence, the Arduino IDE equips you with all the necessary tools to write, compile, and upload code to your Arduino boards. With its intuitive interface and informative feedback mechanisms, it empowers you to unleash your creativity and transform your programming ideas into tangible interactions with the physical world.

- Verify

  This button acts as a code inspector. Clicking it checks your sketch for errors during compilation. Compilation translates your human-readable code into a language the Arduino board understands. Catching errors early with Verify helps ensure your sketch functions as intended when uploaded.

- Upload

  This button compiles your sketch and uploads it to the Arduino board you've selected. Compilation transforms your code into instructions the board can execute. Uploading sends these instructions to the board, preparing it to perform the actions you've programmed.

  Note: If you're using a special tool external programmer to upload code to your board, holding the "shift" key while clicking the Upload button changes the function to "Upload using Programmer.

- New

  This button creates a brand-new sketch for you to start coding! A fresh canvas awaits your programming creativity.

- Open

  Perhaps you have a masterpiece in the works or want to revisit a previous project. Clicking Open displays a menu of all the sketches saved in your sketchbook a designated folder for your Arduino projects. Selecting a sketch from this menu opens it in the current window, replacing any existing code.

  Note: A minor quirk exists in the Open function. Due to a software glitch, the menu might not show all your sketches if you have a large collection. To overcome this, you can access the File menu and navigate to "Sketchbook" to open any sketch from there.

- Save

  Don't lose your hard work! Clicking Save ensures your current sketch is securely stored in your sketchbook.

- Serial Monitor

  This button opens the serial monitor, a powerful communication tool. The serial monitors acts as a bridge between your computer and the Arduino board, allowing you to view text output generated by your sketch or send data commands to the board for interactive projects.

The Arduino IDE offers a robust set of functionalities accessible through five well-organized menus: File, Edit, Sketch, Tools, and Help. These menus are context-sensitive, meaning they dynamically adjust their options based on your current activity within the IDE. Let's explore each menu in detail:

File Menu:

- **New:** Creates a fresh sketch for you to begin coding, providing a clean slate for your programming endeavors.
- **Open:** Opens a file explorer window, allowing you to browse your computer and locate a previously saved sketch to load into the editor.
- **Open Recent:** Provides a convenient shortcut to your most recently accessed sketches, saving you time searching through folders.
- **Sketchbook:** Showcases all the sketches currently residing within your designated sketchbook folder. Clicking on any sketch name opens it in a new editor window.
- **Examples:** Grants access to a comprehensive library of code samples provided by the Arduino software or various libraries. These examples are categorized for easy navigation by topic or library, offering valuable learning resources and starting points for your projects.
- **Close:** Closes the current instance of the Arduino IDE window.
- **Save:** Saves your current sketch under its existing name. If the sketch is unnamed, a "Save as" window will prompt you to provide a name.
- **Save as...:** Allows you to save a copy of the current sketch with a different name, ensuring you can maintain multiple versions or create backups.
- **Page Setup:** Opens a window for configuring printing settings, allowing you to prepare your code for printing if needed.
- **Print:** Sends the current sketch to the printer based on the specifications defined in the Page Setup window.
- **Preferences:** Opens a window where you can customize various settings of the IDE, including the interface language and other preferences.

- **Quit:** Closes all open IDE windows. Notably, any sketches open at the time of quitting will automatically reopen when you launch the IDE again.

Edit Menu:

- **Undo/Redo:** These options provide a safety net, allowing you to reverse or redo editing actions you've performed on your code. Undo lets you go back one or more steps, while Redo brings you back to where you were after using Undo.
- **Cut/Copy/Paste:** This familiar trio offers standard editing functionalities. Cut removes the selected text and stores it in the clipboard, Copy duplicates the selection to the clipboard, and Paste inserts the clipboard content at the cursor's current position in the editor.
- **Copy for Forum/Copy as HTML:** These options cater to sharing your code. "Copy for Forum" prepares the code with syntax highlighting, making it suitable for pasting into online forums while preserving formatting. "Copy as HTML" copies the code as HTML code, allowing you to embed it in web pages.
- **Select All:** Highlights the entire content of the editor window, making it easy to perform actions on the whole sketch.
- **Comment/Uncomment:** Toggles comments on or off for selected lines of code. Adding comments starting with // makes your code more readable for yourself and others, while removing comments makes the code executable.
- **Increase/Decrease Indent:** Maintains code readability by adjusting indentation levels. Increase Indent adds a space at the beginning of each selected line, shifting the text right, while Decrease Indent removes a space, shifting the text left.
- **Find/Find Next:** The Find function opens a window where you can specify text to search for within the current sketch. You can refine your search with various options. "Find Next" highlights the next occurrence of the specified search term relative to the cursor's position.

Sketch Menu:

- **Verify/Compile (✓):** This dynamic duo checks your code for errors. The Verify function compiles your sketch, translating it into a language the Arduino board understands. During compilation, it also reports memory usage for your code and variables in the console area, helping you optimize your sketch if necessary.

- **Upload (↑):** Ready to bring your code to life? This button compiles your sketch and uploads the compiled binary file onto the Arduino board you've selected through the configured port. This is when the magic happens – your code starts controlling the hardware!
- **Upload Using Programmer:** This option is for advanced users. It uploads the code using an external programmer instead of the Arduino board's built-in USB-to-serial connection. However, be cautious! Using this function overwrites the bootloader on the board. You'll need to use "Tools > Burn Bootloader" to restore it and regain the ability to upload via USB. This method offers the benefit of utilizing the board's full flash memory capacity for your sketch.

- **Export Compiled Binary:** This functionality allows you to save a .hex file containing the compiled binary code of your sketch. This file can serve as an archive or be uploaded to the board using other tools.
- **Show Sketch Folder:** Need to locate the folder where your sketch is stored on your computer? This option opens the folder containing the current sketch, providing easy access to manage related files.
- **Include Library:** The Arduino IDE leverages libraries to extend functionality. This menu item helps you incorporate libraries into your sketch by inserting the necessary #include statements at the beginning of your code. Additionally, you can access the Library Manager from here to import new libraries from compressed .zip files.
- **Add File...:** Want to incorporate additional source files into your sketch? This option allows you to add a source file from another location on your computer. The added file appears as a new tab within the sketch window, enabling you to manage multiple code files within the same project. You can remove unnecessary files using the tab menu accessible by clicking the small triangle icon next to the serial monitor icon on the right side of the toolbar.

Tools Menu:

- **Auto Format:** This feature is your code's beautician! It automatically formats your code, enhancing readability by indenting it consistently. Proper indentation aligns opening and closing curly braces, and statements within braces are further indented, making your code visually organized and easier to understand.
- **Archive Sketch:** Ever need to create a backup of your project? This option creates a compressed archive .zip format of your current sketch. The archive is placed in the same directory as the original sketch for easy retrieval.

- **Fix Encoding & Reload:** This option addresses potential character encoding issues that might arise due to differences between your editor's character map and other operating systems. It attempts to fix any discrepancies, ensuring your code is interpreted correctly.
- **Serial Monitor:** This familiar button opens the serial monitor window, a powerful communication channel. You can use it to view text output generated by your sketch running on the Arduino board or send data commands to the board for interactive projects. Opening the serial monitor often resets the board if it supports resetting via the serial port.
- **Board:** This crucial menu allows you to select the specific Arduino board you're using. Different boards have varying capabilities and require the IDE to be configured accordingly. Selecting the correct board ensures the code you upload is compatible with your hardware.
- **Port:** This menu displays all the serial devices connected to your computer. The IDE should automatically refresh this list whenever you open the Tools menu. Selecting the correct port ensures the IDE communicates with the right Arduino board.
- **Programmer:** This menu caters to advanced users who employ external hardware programmers to upload code to boards or chips instead of the standard USB-to-serial connection. In most cases, you won't need this option. However, it becomes relevant when burning a bootloader onto a new microcontroller, which typically comes without a bootloader pre-installed.
- **Burn Bootloader:** The options within this menu enable you to burn a bootloader onto the microcontroller of your Arduino board. A bootloader is a small program that allows the IDE to upload code to the board. While not required for everyday use, bootloader burning becomes necessary if you purchase a new microcontroller. Before burning the bootloader,

Help Menu:

The Arduino IDE's Help menu offers a wealth of resources at your fingertips, all accessible without an internet connection. Here's a breakdown of what it provides:

- **Offline Documentation:** Forget scrambling for internet access! The Help menu stores local copies of essential Arduino software documents, including:
    - **Getting Started:** A beginner-friendly guide to kickstart your Arduino journey.
    - **Reference:** A comprehensive reference manual containing detailed information on functions, commands, and keywords used in Arduino programming.
    - **IDE Guide:** This very guide you're reading! It offers in-depth explanations of the IDE's functionalities and menus.

- **Other Documents:** The Help menu might contain additional documentation specific to your Arduino software version.

- **Find in Reference:** This is the magic trick of the Help menu. Place your cursor over any function or command within your code editor, and then navigate to the Help menu. Select "Find in Reference," and voila! The IDE will automatically direct you to the relevant page in the local reference manual, providing detailed information about the element under your cursor. This interactive feature streamlines your learning process by offering instant access to relevant documentation.

Key Takeaways:

- The Help menu serves as your offline library for essential Arduino software documentation.
- "Find in Reference" acts as a contextual guide, instantly directing you to the part of the reference manual that explains the code element you're working with.By effectively utilizing the Help menu, you can leverage the wealth of knowledge available within the Arduino IDE and enhance your coding experience.

7.2 Installation of ESP32 in Arduino IDE

Step 1: Prerequisites: Arduino IDE Installed

Before starting this installation procedure, make sure you have the latest version of the Arduino IDE installed in your computer. If you don‟t, uninstall it and install it again. Otherwise, it may not work. the latest Arduino IDE software installed from arduino.cc/en/Main/Software, continue with this tutorial.

Step 2: Installing ESP32 Add-on in Arduino IDE

Step 3:

Enter Https://dl.espressif.com/dl/package_esp32_index.j... Into the “Additional Board Manager URLs” Field As Shown in the Figure Below. Then, Click the “OK” Button:
Note: if you already have the ESP8266 boards URL, you can separate the URLs with a comma asfollows:
https://dl.espressif.com/dl/package_esp32_index.json,http://arduino.esp8266.com/stable/package_esp8266com_index.json

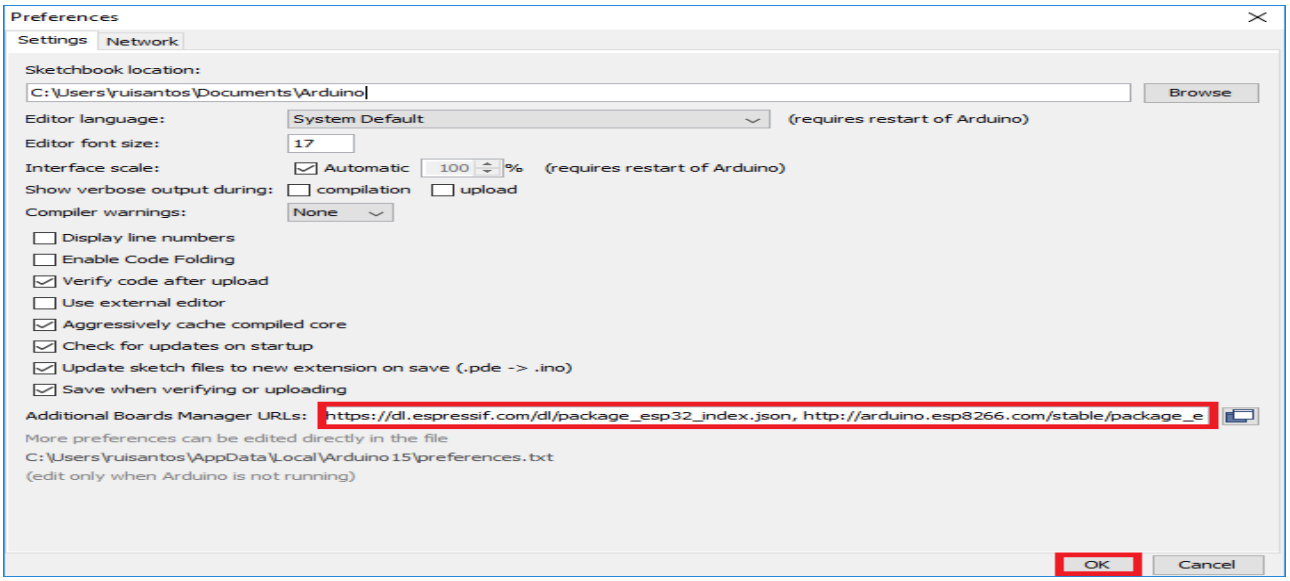


Figure 7.2.1: Configuration of Additional Board Manager URL for ESP32 in Arduino IDE

Step 4: Open the Boards Manager. Go to Tools > Board > Boards Manager…

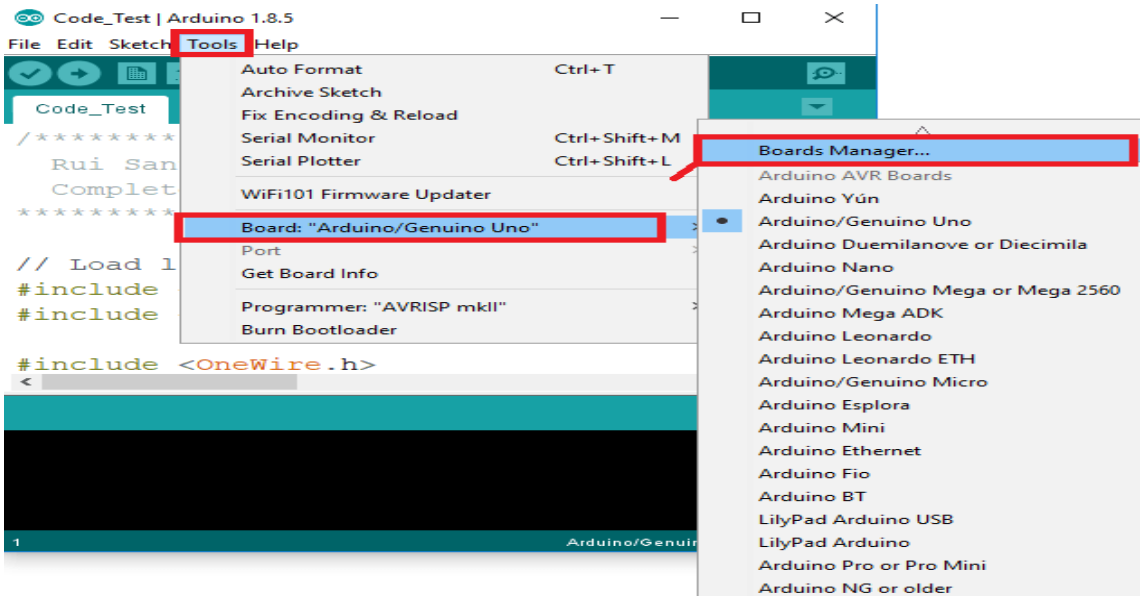


Figure7.2.2: Accessing the Boards Manager in Arduino Software

Step 5:

Search for ESP32 and Press Install Button for the “ESP32 by Espressif Systems“:

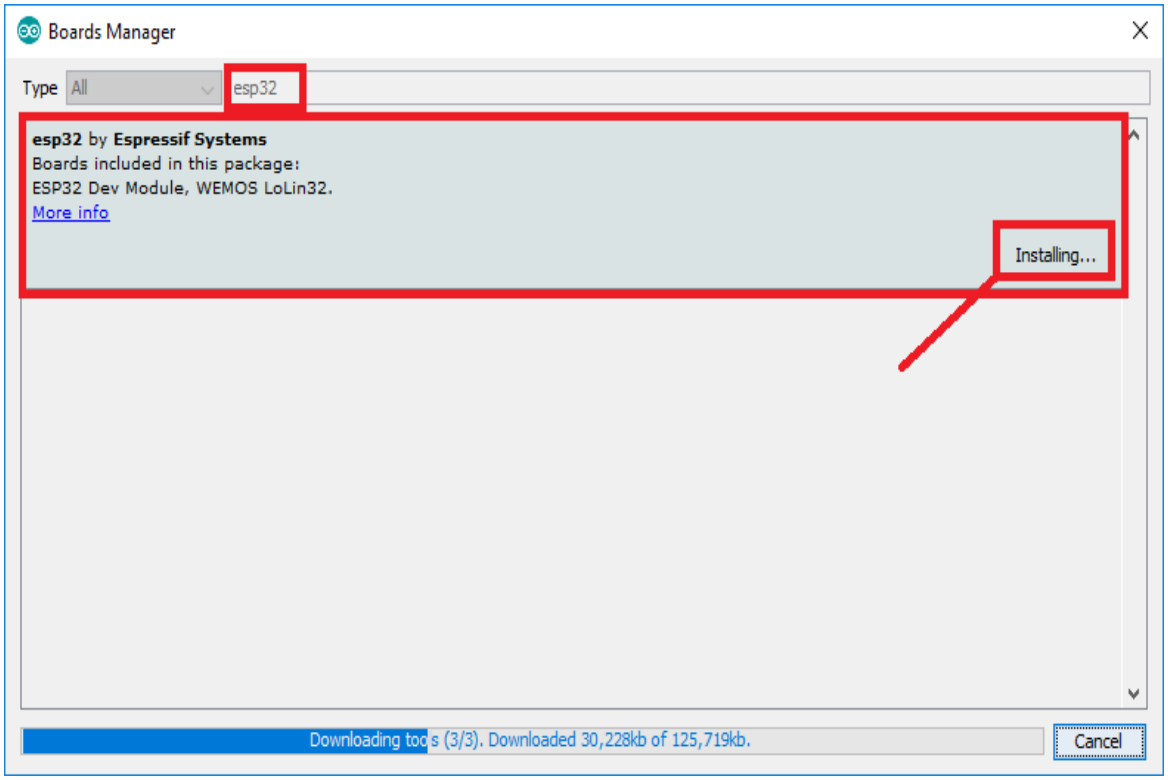


Figure7.2.3: ESP32 Board Installation Via Arduino Boards Manager

Step 6: That's It. It Should Be Installed After a Few Seconds.

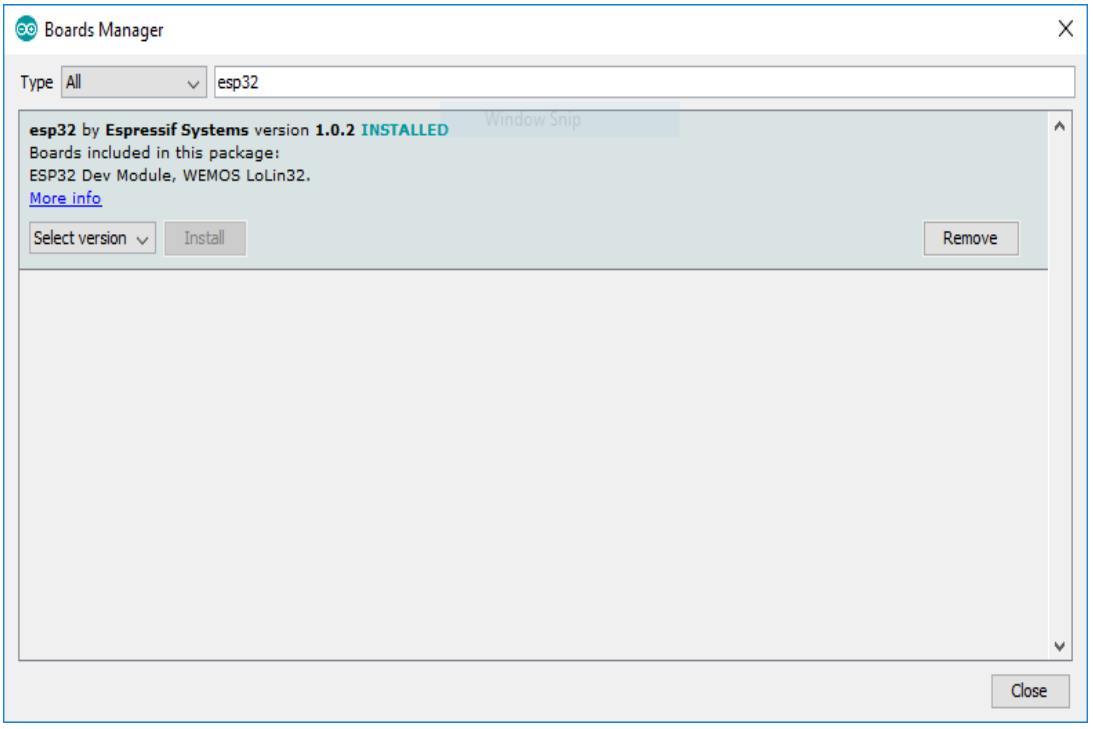


Figure7.2.4 : Conformation of ESP32 Installation in Board Manager

Step 7: Testing the Installation

Selecting ESP32 Board (DOIT ESP32 DEVKIT V1) in Arduino IDE Plug the ESP32 board to your computer. With your Arduino IDE open, follow these steps:Select your Board in Tools > Board menu (in my case it‟s the DOIT ESP32 DEVKIT V1) Now go to **Tools → Port**.Select the **COM port** where the ESP32 is connected (e.g., COM3, COM5).Keep other settings at default (Upload Speed, Flash Frequency, etc.). The ESP32 board is now **ready for programming** in Arduino IDE. You can now **write, compile, and upload** your ESP32 code successfully.

7.3 ThingSpeak IoT Platform

ThingSpeak is IoT platform for user to gather real-time data; for instance, climate information, location data and other device data. In different channels in ThingSpeak, you can summarize information and visualize data online in charts and analyze useful information. ThingSpeak can integrate IoT:bit (micro:bit) and other software/ hardware platforms. Through IoT:bit, you can upload sensors data to ThingSpeak (e.g. temperature, humidity, light intensity, noise, motion, raindrop, distance and other device information).

Step 1

Go to https://thingspeak.com/, register an account and login to the platform

Step 2

Choose Channels -> My Channels -> New Channel

Step 3

Input Channel name, Field1, then click “Save Channel

Step 4

chat for data field1

Step 5

Open your web browser, go to https://thingspeak.com, select your channel > “API Keys” copy the API key as follows

7.4 Installation and Running of Python Application

Step 1: Download Python

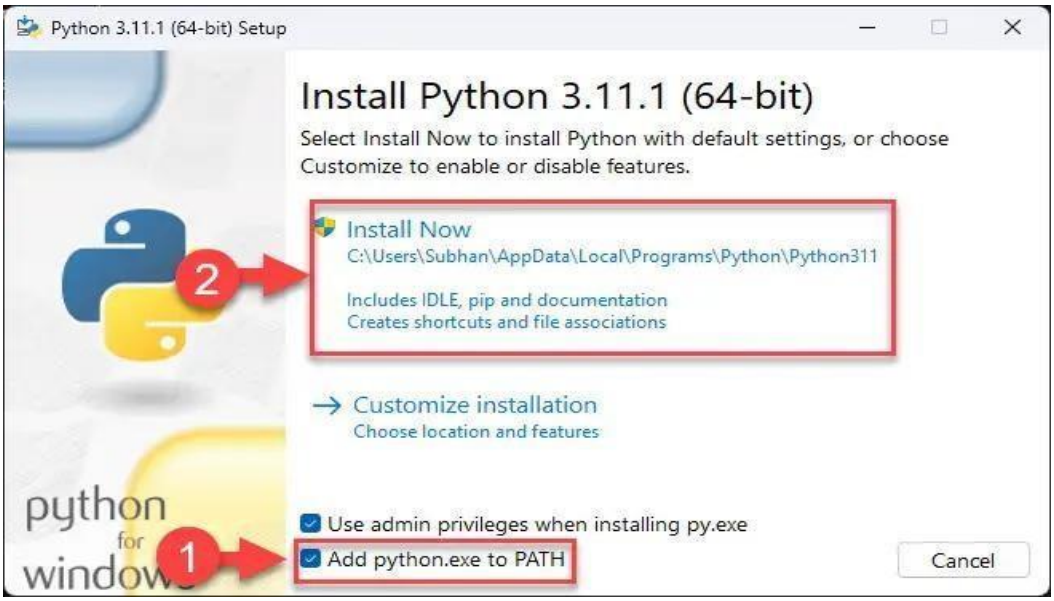


Figure7.4.1: Python Installation Setup Window

Visit the official website of Python (https://www.python.org). Click on the Downloads section and download the latest version of Python suitable for your operating system (Windows, macOS, or Linux). Always download Python from the official website to ensure security and compatibility.

Step 2: Install Python

Open the downloaded installer file. Before clicking install, make sure to check the option "Add Python to PATH". Then click on Install Now and wait for the installation process to complete. After successful installation, a confirmation message will appear.

Step 3: Verify Python Installation

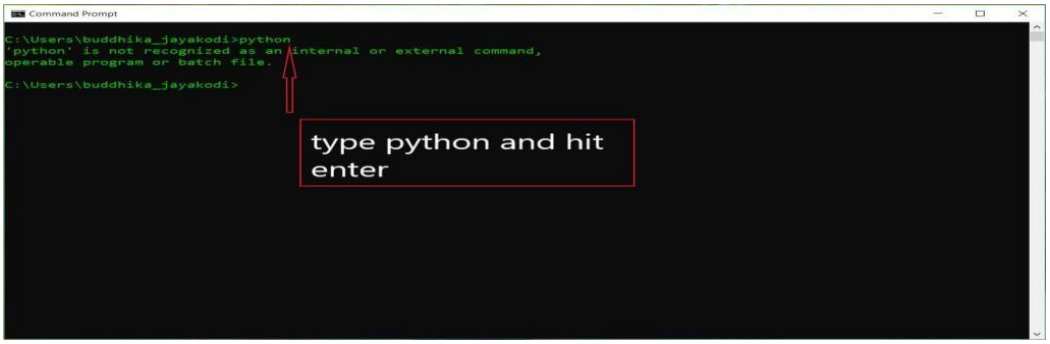


Figure7.4.2:Command Prompt

Open Command Prompt (Windows) or Terminal (macOS/Linux). Type the following command:

python --version

If Python is installed correctly, the installed version (e.g., Python 3.x.x) will be displayed. This confirms that Python is successfully installed on your system**.**

Step 4: Install a Code Editor

You can write Python programs using editors like IDLE (comes with Python) or external editors such as Visual Studio Code or PyCharm. These editors provide better features like syntax highlighting, debugging, and auto-completion.

Step 5: Write and Run Python Program

Create a new file and write a simple Python program:

The output will be displayed on the screen. This confirms that your Python application is running successfully.

Role of Python in the Application

Python serves as the core programming language of the application. It handles all backend operations such as data processing, model training, prediction, and API communication. The logic written in Python controls how data flows from input to output.

The Python script performs the following tasks:

- Loads the dataset
- Trains the machine learning model
- Accepts user or IoT input
- Processes the data
- Generates predictions
- Displays results on the interface

Thus, Python acts as the brain of the system, making decisions and controlling functionality.

Machine Learning Integration

Random Forest Algorithm ,The application uses a Machine Learning technique from the field of Machine Learning, specifically the Random Forest algorithm. This algorithm consists of multiple decision trees that work together to make accurate predictions.

Each decision tree analyzes the input data independently and produces an output. The final prediction is determined by majority voting among all trees. This approach improves accuracy and reduces errors caused by overfitting. The model is trained using a dataset containing health parameters and corresponding conditions (good or bad). After training, the model can predict the condition for new input data provided by the user or IoT system.

Data Processing and Training The dataset is loaded using the Pandas library and processed to convert categorical values into numerical form. The data is then divided into training and testing sets using the train-test split method.The training data is used to teach the model patterns, while the testing data is used to evaluate its performance. The accuracy score is calculated to measure how well the model predicts outcomes.This process ensures that the system provides reliable and consistent predictions.API and IoT Data Integration, Auto Mode – Live Data Fetching The application includes an Auto Mode that retrieves real-time data from the ThingSpeak cloud platform using an API. The API sends a request to the server and receives data in JSON format. The Python requests library is used to fetch this data, which includes: Heart rate ,SpO_2 ,Body temperature ,Room temperature .This data is then processed and passed to the machine learning model for prediction. This feature enables real-time health monitoring without manual input. Manual Mode – User Input. In Manual Mode, the user enters values directly through the interface. These values are collected using input fields provided by Stream lit Once the user clicks the Predict button, the input data is converted into a numerical array and sent to the machine learning model. The model then analyzes the data and returns the prediction.

This mode is useful for testing and demonstration purposes when IoT data is not available. Interface (UI) Explanation, Overview of the Interface The user interface of the application is built using Streamlit, which transforms Python scripts into interactive web applications. The interface is designed with a modern and visually appealing layout using custom CSS styling.

It includes a dark-themed background, a transparent panel with a blur effect, and color-coded outputs. The design enhances readability and provides a professional healthcare look. The top section of the interface displays the application title “MEDTEST TOOL” along with a subtitle. This section introduces the application and gives it a clear identity. The use of large fonts and bright colors makes the title visually prominent and engaging for users The interface includes a mode selection feature implemented using radio buttons. Users can choose between Auto Mode and Manual Mode.

This section is important because it allows flexibility in how data is provided to the system. It ensures that the application can work both with live IoT data and manual inputs. Auto Mode Display Section In Auto Mode, the interface displays real-time sensor values fetched from the ThingSpeak API. These values are shown in a clean and readable format using text elements. This section provides live monitoring capability, making the system dynamic and interactive. Manual Input SectionIn Manual Mode, the interface provides input fields where users can enter health parameters. These fields are created using number input widgets. A Predict button is included to trigger the prediction process. Once clicked, the application processes the input data and displays the result.

CHAPTER-8

RESULTS AND DISCUSSIONS

CHAPTER 8

RESULTS AND DISCUSSIONS

8.1 Testing by Using LCD:

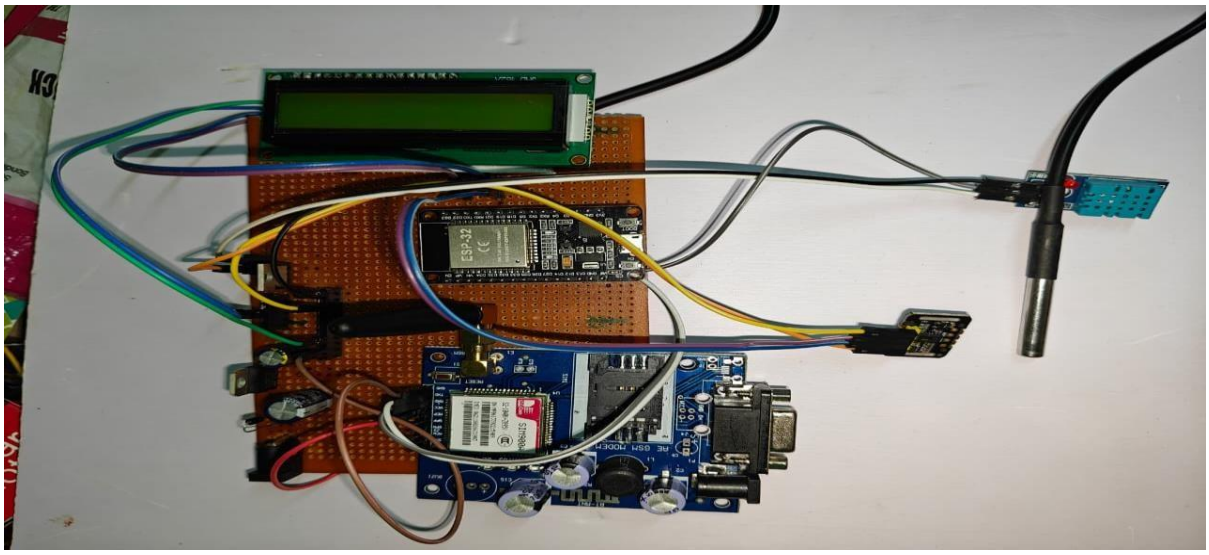


Figure 8.1.1 : Hardware Setup of Smart Health Monitoring System

The first image shows the complete hardware setup of the IoT-based health monitoring system built on a PCB board. It includes an ESP32 microcontroller, LCD display, temperature sensor, SpO_2/heart rate sensor, and a GSM module. All components are interconnected using wires, and the system is designed to collect real-time physiological data from the patient. The ESP32 acts as the main controller, processing sensor data and enabling communication with cloud platforms and mobile networks.



Figure 8.1.2: LCD Display – Alert Notification

The second image shows the LCD displaying an alert message “BODY TEMP HIGH”. This indicates that the system continuously monitors the patient’s body temperature

and generates warnings when abnormal conditions are detected. Such real-time alerts help in early detection of health issues and allow immediate action to prevent critical situations.



Figure 8.1.3 LCD Display – Real-Time Health Parameters

The third image displays real-time health parameters such as Heart Rate (HR), SpO_2 (oxygen level), Body Temperature (BT), and Humidity (H). The LCD provides instant feedback to the user, allowing continuous monitoring without needing external devices. This feature is useful for both patients and healthcare providers to track vital signs easily.

8.2 Testing by THINGSSPEAK

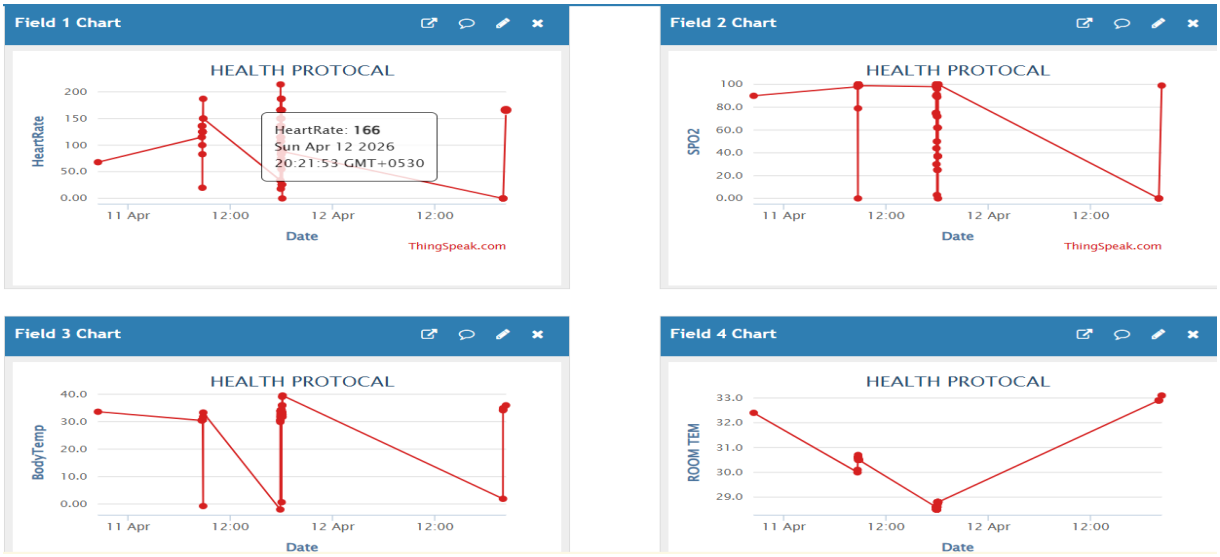


Figure 8.2 ThingSpeak Cloud Data Visualization

The fourth image shows graphical representation of health data on the ThingSpeak IoT platform. Parameters like heart rate, SpO_2, body temperature, and room temperature are

plotted over time. This enables remote monitoring, historical data analysis, and better decision-making by doctors using cloud-based access.

8.3 Testing by ML MODEL

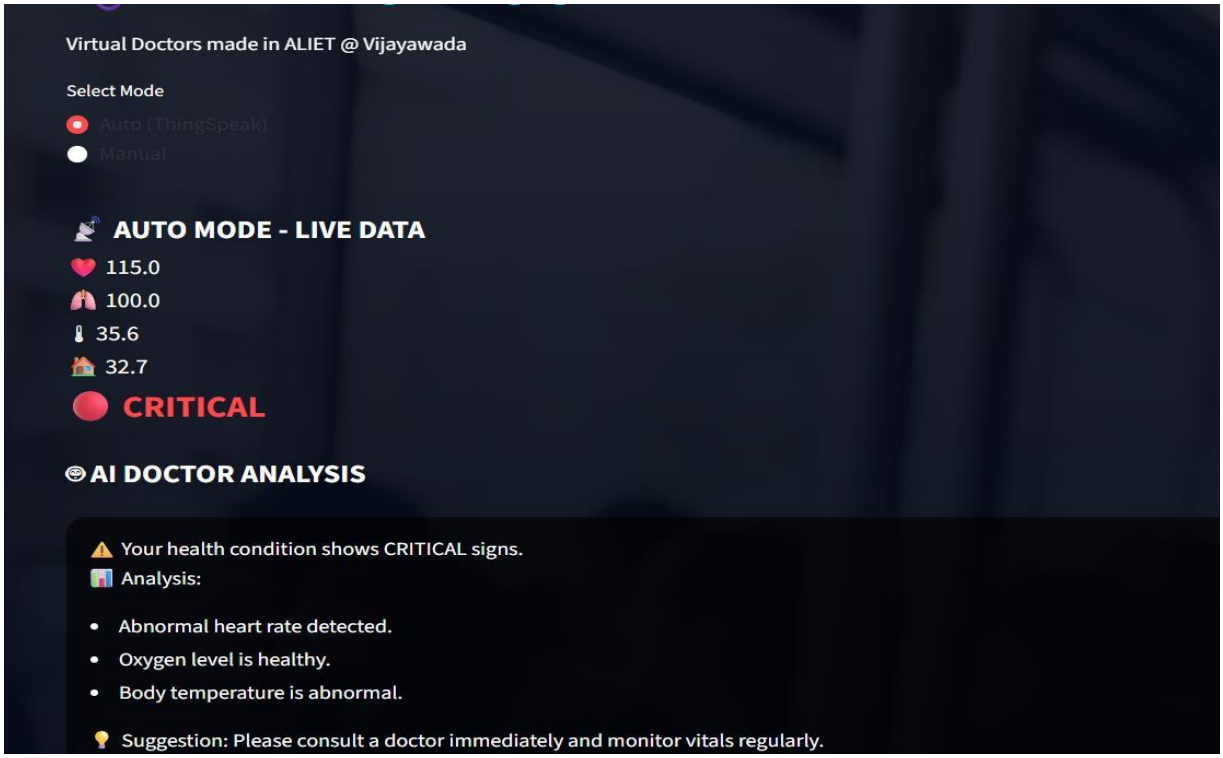


Figure 8.3: Patients Condition checking through ml model

The fifth image represents an AI-powered dashboard that analyzes patient data and classifies the condition (e.g., Critical). It provides insights such as abnormal heart rate or temperature and suggests actions. This intelligent system enhances healthcare by offering automated diagnosis and decision support.

8.4 ALEART MSG

The sixth image shows an SMS alert sent to a mobile phone containing critical health parameters like body temperature, heart rate, and SpO_2 levels. This feature ensures that caregivers or doctors are immediately informed during emergencies, enabling quick response and improving patient safety.

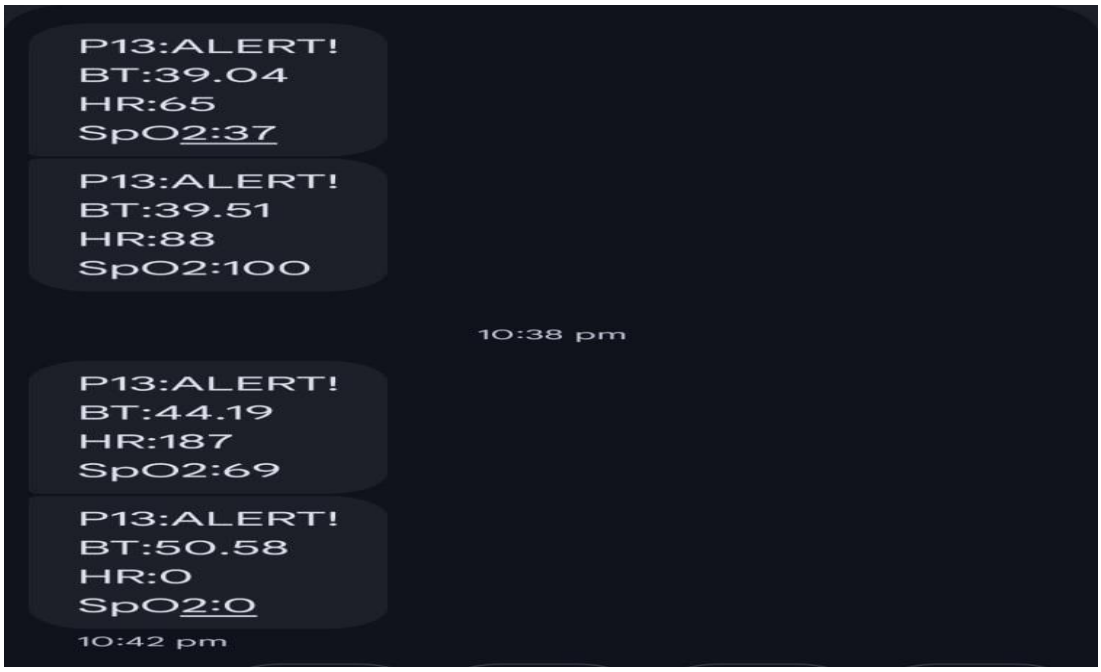


Figure 8.4: Alert SMS

CHAPTER-9

ADVANTAGES AND APPLICATIONS

CHAPTER 9

ADVANTAGES AND APPLICATIONS

Advantages:

1. Real-time Health Monitoring – The system continuously monitors vital parameters such as heart rate, blood oxygen level (SpO_2), and body temperature.
2. Remote Patient Monitoring – Health data can be accessed remotely through IoT cloud platforms, allowing doctors and caregivers to monitor the patient from anywhere.
3. Early Detection of Health Problems – Abnormal health conditions can be identified quickly, helping in early medical intervention.
4. Low Cost and Portable – The system uses affordable sensors and components, making it suitable for home healthcare applications.
5. Multiple Parameter Monitoring – The system monitors several health parameters simultaneously, providing a more complete health analysis.
6. Real-time Display of Data – Health parameters are displayed on the LCD screen, allowing immediate observation by patients or caregivers.
7. Cloud Data Storage – Health data is stored in the cloud, enabling long-term monitoring and historical data analysis.
8. Alert System – The system can send alerts or notifications when abnormal health conditions are detected.
9. Improved Healthcare Accessibility – Useful for elderly patients and people living in remote areas who require continuous monitoring.
10. Integration with Machine Learning – Machine learning algorithms can analyze health data and classify patient conditions for better prediction and diagnosis.

Applications:

- Integration with Ambulance and Emergency Services
- Support for Cardiac Rehabilitation Programs
- Contribution to Clinical Trials and Research
- Implementation in Elderly Care Facilities
- Utilization in Sports Medicine and Athletic Training
- Integration with Telemedicine and Remote Consultations
- Use Cases in Military and Aerospace Medicine
- Application in Community Health Initiatives
- Home Monitoring for Heart Patients
- Hospital Monitoring and Post-Surgery Care

CONCLUSION

The developed IoT and Machine Learning based Smart Health Monitoring System successfully demonstrates an efficient and reliable solution for real-time patient health monitoring. By integrating multiple sensors such as MAX30102, DS18B20, and DHT11 with a microcontroller, the system is capable of continuously measuring vital parameters including heart rate, blood oxygen saturation (SpO_2), body temperature, and environmental conditions. The real-time display through an LCD ensures that immediate health information is available to the user without delay.

The integration of IoT technology using ESP32 enables seamless transmission of collected data to the cloud platform (ThingSpeak), allowing remote monitoring from anywhere in the world. This feature significantly enhances accessibility and makes the system highly suitable for home healthcare, elderly monitoring, and rural areas where medical facilities may be limited. The storage of data with timestamps also supports historical analysis and long-term health tracking.

A major highlight of the project is the incorporation of Machine Learning, which adds intelligence to the system. Instead of relying only on fixed threshold values, the ML model analyzes multiple health parameters simultaneously and classifies the patient's condition as normal or critical. This improves the accuracy of predictions and enables early detection of potential health risks. Additionally, the AI-based explanation system provides meaningful insights and suggestions, making the system more user-friendly and informative.

The system also includes a GSM-based alert mechanism, which ensures immediate response during emergency conditions. When abnormal values are detected, the system automatically sends SMS alerts and makes calls to caregivers or medical professionals. This real-time alert feature enhances patient safety and reduces the risk of delayed medical attention.

Overall, the project achieves its objective of developing a low-cost, portable, and intelligent healthcare monitoring system. It combines embedded systems, IoT, machine learning, and communication technologies into a single integrated solution. The system is scalable and can be further enhanced with additional sensors, advanced AI models, and mobile applications. Hence, it provides a strong foundation for future smart healthcare systems and contributes to improving the quality and accessibility of healthcare services.

FUTURE SCOPE

The future scope of Heart Patient Health Monitoring Systems using IoT is promising, with advancements expected in several key areas:

1. **Integration with Advanced Machine Learning Models**

   The system can be improved by using advanced machine learning or deep learning algorithms to provide more accurate prediction and early detection of diseases.

2. **Mobile Application Development**

   A dedicated mobile application can be developed to allow doctors and caregivers to monitor patient health parameters easily through smartphones.

3. **Addition of More Medical Sensors**

   Additional sensors such as ECG sensors, blood pressure sensors, and glucose monitoring sensors can be integrated to provide more comprehensive health monitoring.

REFERENCES

[1] Wireless Medical Technologies: A Strategic Analysis of Global Markets [online]. International Telecoms Intelligence.

[2] G. Y. Jeong, K. H. Yu, and Kim.N. G. Continuous blood pressure monitoring using pulse wave transit time. In International Conference on Control, Automation and Systems (ICCAS), 2005.

[3] K. Hung, Y. T. Zhang, and B. Tai. Wearable medical devices for tele home healthcare. In Procs.26th Annual International Conference on the IEEE EMBS, 2004.

[4] Fang, Xiang et al: An extensible embedded terminal platform for wireless telemonitoring, Information and Automation (ICIA), 2012 International Conference on Digital Object Identifier: 10.1109/ICInfA.2012.6246761 Publication Year: 2012, Page(s): 668 - 673.

[5] Majer, L., Stopjaková, V., Vavrinský, E.: Sensitive and Accurate Measurement Environment for Continuous Biomedical Monitoring using Microelectrodes. In: Measurement Science Review. - ISSN 1335- 8871. - Vol. 7, Section 2, No. 2 (2007), s. 20-24.

[6] Majer, L., Stopjaková, V., Vavrinský, E.: Wireless Measurement System for Non-Invasive Biomedical Monitoring of Psycho Physiological Processes. In: Journal of Electrical Engineering. - ISSN 1335-3632. - Vol. 60, No. 2 (2009), s. 57-68.

[7] Goutam Motika, Abinash Prusty," Wireless Fetal Heartbeat Monitoring System Using ZigBee & IEEE 802.15.4 Standard", 2011 Second International Conference on Emerging Applications of Information Technology, 978-0-7695 4329-1/11 $26.00 © 2011 IEEE DOI 10.1109/EAIT.2011.89.

[8] Warsuzarina Mat Jubadi, Siti Faridatul Aisyah Mohd ahak", Heartbeat Monitoring Alert via SMS", 978-1-4244-4683-4/09/$25.00 ©2009 IEEE.

[9] Ovidiu Apostu, Bogdan Hagiu, Sever Paşca, Wireless ECG Monitoring and Alarm System Using ZigBee"2011 The International Symposium on ADVANCED TOPIC INELECTRICAL ENGINEERING2068-7966/ATEE 2011.

IJARCCE ISSN (O) 2278-1021, ISSN (P) 2319-5940



International Journal of Advanced Research in Computer and Communication Engineering

Impact Factor 8.471 ⁜ Peer-reviewed & Refereed journal ⁜ Vol. 15, Issue 4, April 2026

DOI: 10.17148/IJARCCE.2026.15470

IoT and Machine Learning Based Smart Health Monitoring System

Sidagam Geethika[1], P. Bose Babu[2]

B.Tech Student, Department of Electronics and Communication Engineering,

Andhra Loyola Institute of Engineering and Technology (ALIET), Vijayawada, India[1]

Assistant Professor, Department of Electronics and Communication Engineering,

Andhra Loyola Institute of Engineering and Technology (ALIET), Vijayawada, India[2]

Abstract: The rapid advancement of Internet of Things (IoT) and Machine Learning (ML) technologies has opened new avenues for developing intelligent and cost-effective healthcare monitoring systems. This paper presents the design and implementation of an IoT and Machine Learning based Smart Health Monitoring System capable of continuously monitoring multiple physiological and environmental parameters in real time. The proposed system integrates the MAX30102 sensor for heart rate and blood oxygen saturation (SpO_2) measurement, the DS18B20 sensor for body temperature monitoring, and the DHT11 sensor for ambient temperature and humidity sensing. An ESP32 microcontroller acts as the central processing unit, collecting sensor data and transmitting it to the ThingSpeak cloud platform via Wi-Fi for remote access and visualization. A 16×2 I2C LCD display provides immediate local readout of health parameters. A Random Forest machine learning algorithm, deployed through a Streamlit-based Python application, classifies the patient's health condition as Normal or Critical. Additionally, a GSM module (SIM900A) sends automated SMS alerts and voice calls to caregivers when critical conditions are detected. Experimental results demonstrate that the system achieves reliable real-time monitoring, accurate ML-based health classification, and effective emergency notification. The system is low cost, portable, and scalable, making it highly suitable for home healthcare, elderly monitoring, and remote medical applications.

Keywords: Internet of Things (IoT), Machine Learning, Health Monitoring System, ESP32, MAX30102, ThingSpeak, Random Forest, Remote Patient Monitoring, GSM Alerts.

I. INTRODUCTION

The increasing prevalence of chronic diseases and sudden cardiac events has underscored the critical need for continuous, real-time patient health monitoring systems. Traditional healthcare relies heavily on in-hospital observation, which is not always feasible for elderly patients, individuals with chronic conditions, or those residing in remote regions. The convergence of Internet of Things (IoT) and Machine Learning (ML) technologies offers a transformative solution by enabling intelligent, remote, and autonomous health monitoring outside clinical settings.

The Internet of Things refers to a network of interconnected physical devices embedded with sensors, software, and communication capabilities that collect and exchange data over the internet [1]. IoT-based health monitoring systems leverage low-cost biomedical sensors to capture vital physiological parameters such as heart rate, blood oxygen saturation (SpO_2), and body temperature. These parameters are then transmitted to cloud platforms for storage, visualization, and remote access by healthcare professionals. With the integration of Machine Learning algorithms, such systems gain the ability to intelligently analyze health data, identify abnormal patterns, and predict potential health risks, thereby enabling early medical intervention.

Heart diseases remain one of the leading causes of mortality worldwide. Conditions such as coronary artery disease, arrhythmias, and heart failure require continuous monitoring for effective management. A smart health monitoring system that integrates multiple sensors with ML-based analysis can assist doctors and caregivers in tracking patient health trends and responding promptly to emergencies.

The system proposed in this paper integrates the ESP32 microcontroller, the MAX30102 pulse oximetry sensor, the DS18B20 body temperature sensor, and the DHT11 ambient sensor to form a comprehensive health monitoring platform. The collected data is visualized locally on an LCD display and transmitted to the ThingSpeak cloud for remote monitoring. A Random Forest ML classifier, implemented in Python using the Streamlit framework, provides automated health condition classification. In critical cases, the integrated GSM module dispatches SMS alerts and voice calls to designated emergency contacts, ensuring timely medical response.

This paper is organized as follows: Section II presents a literature survey of related work. Section III describes the system scope and design objectives. Section IV details the hardware and software methodology. Section V presents the experimental results and discussion. Section VI provides the conclusion, and Section VII outlines future work.

IJARCCE ISSN (O) 2278-1021, ISSN (P) 2319-5940

International Journal of Advanced Research in Computer and Communication Engineering

Impact Factor 8.471 ⁜ Peer-reviewed & Refereed journal ⁜ Vol. 15, Issue 4, April 2026

DOI: 10.17148/IJARCCE.2026.15470

II. Literature Survey

A growing body of research has explored the integration of IoT and machine learning technologies in healthcare monitoring systems. The following summarizes key relevant works.

Gupta et al. [1] developed an IoT-based smart healthcare monitoring system utilizing wearable sensors to continuously monitor vital signs, demonstrating that IoT-enabled wearables can effectively support remote patient monitoring and early detection of health anomalies. Khan et al. [2] proposed a real-time IoT system using cloud computing for patient health monitoring, illustrating how cloud-based architectures facilitate data storage, real-time alerts, and accessibility for healthcare providers.

Patel and Shah [3] combined IoT technology with machine learning algorithms to build an intelligent healthcare monitoring system, proving that ML-based analysis significantly enhances disease detection accuracy compared to fixed threshold-based methods. Chatterjee et al. [4] introduced an AI-enabled IoT framework for remote healthcare monitoring published in IEEE Access, emphasizing that artificial intelligence can process large health datasets to provide accurate early diagnoses. Ramesh and Prakash [5] explored IoT healthcare monitoring with edge computing to reduce communication latency and enable faster abnormality detection, extending the applicability of IoT health systems to resource-constrained environments.

The present work advances beyond these contributions by integrating multiple sensor modalities (pulse oximetry, body temperature, ambient monitoring) with a Random Forest ML classifier and GSM-based emergency alerting into a unified, low-cost, portable system.

III. Methodology

A. System Architecture and Block Diagram

The proposed smart health monitoring system is organized into five functional layers: (1) Sensor Layer, (2) Processing Layer, (3) Local Display Layer, (4) Cloud Communication Layer, and (5) Machine Learning and Alert Layer. The ESP32 microcontroller acts as the hub integrating all layers. Fig. 1 illustrates the overall block diagram of the system.

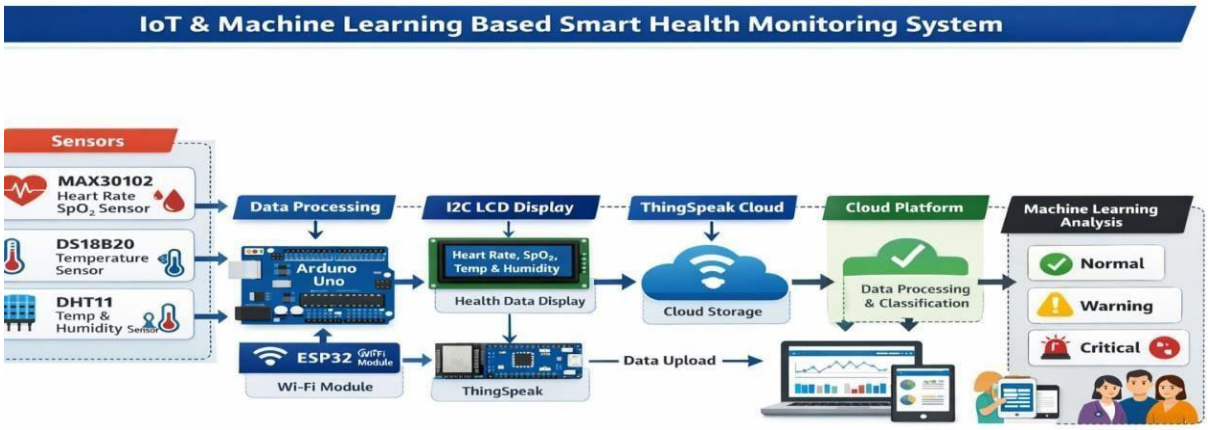


Fig. 1: Block Diagram of the IoT and ML Based Smart Health Monitoring System

The MAX30102, DS18B20, and DHT11 sensors connect to the ESP32 via I2C and GPIO interfaces. The ESP32 processes the sensor data, displays it on the I2C LCD, and transmits it to the ThingSpeak cloud using Wi-Fi. The Python-based Streamlit application fetches data from ThingSpeak via an API and runs the trained Random Forest model to classify the patient's health status. In critical conditions, the GSM module (SIM900A) sends SMS alerts and makes voice calls.

B. Hardware Components and Pin Specifications

The hardware platform consists of the following components:

1) ESP32 Microcontroller: The ESP32 is a dual-core, 240 MHz microcontroller with built-in Wi-Fi and Bluetooth, developed by Espressif Systems. It serves as the central processing unit of the system, interfacing with all sensors and communication modules. Key pin assignments include: 3.3V for sensor supply, GND for common ground, GPIO21 (SDA) and GPIO22 (SCL) for I2C communication, GPIO4 for the DHT11 data line, GPIO5 for the DS18B20 one-wire data line, GPIO16 (RX) connected to GSM TX, and GPIO17 (TX) connected to GSM RX.

IJARCCE ISSN (O) 2278-1021, ISSN (P) 2319-5940

International Journal of Advanced Research in Computer and Communication Engineering

Impact Factor 8.471 ⁜ Peer-reviewed & Refereed journal ⁜ Vol. 15, Issue 4, April 2026

DOI: 10.17148/IJARCCE.2026.15470

2) MAX30102 (Pulse Oximetry and Heart Rate Sensor): The MAX30102 is an integrated biosensor module that measures heart rate and blood oxygen saturation (SpO_2) using infrared and red LED photoplethysmography. It communicates with the ESP32 via the I2C protocol (SDA: GPIO21, SCL: GPIO22) and operates at 3.3 V.

3) DS18B20 (Body Temperature Sensor): The DS18B20 is a digital one-wire temperature sensor with a measurement range of -55°C to +125°C and ±0.5°C accuracy, suitable for body temperature monitoring. It connects to GPIO5 and requires a 4.7 kΩ pull-up resistor between the data line and VCC (3.3 V).

4) DHT11 (Ambient Temperature and Humidity Sensor): The DHT11 monitors room temperature (0°C to 50°C) and relative humidity (20% to 90% RH). It uses a single-wire digital communication protocol connected to GPIO4 and is powered at 3.3 V. Ambient monitoring complements physiological measurements and enhances ML-based health classification.

5) 16×2 I2C LCD Display: The LCD display (with an I2C backpack module at address 0x27) provides real-time local readout of health parameters. Using I2C reduces the wiring to four pins (VCC: 5 V, GND, SDA: GPIO21, SCL: GPIO22), sharing the I2C bus with the MAX30102 sensor.

6) SIM900A GSM/GPRS Module: The SIM900A GSM module enables cellular network communication for SMS alerts and voice calls in emergency conditions. It operates via UART serial communication (TX to GPIO17, RX to GPIO16) using AT commands. The module requires a stable external power supply capable of delivering up to 2 A.

TABLE I

Sensor Measurement Ranges and Critical Thresholds

Sensor	Parameter	Measurement Range	Normal Range	Critical Condition
MAX30102	Heart Rate	30 – 220 bpm	60 – 100 bpm	< 50 or > 110 bpm
MAX30102	SpO_2	0 – 100 %	95 – 100 %	< 90 %
DS18B20	Body Temperature	-55°C to 125°C	36 – 37.5°C	> 38.5°C or < 35°C
DHT11	Room Temperature	0°C – 50°C	20 – 30°C	< 15°C or > 35°C
DHT11	Humidity	20% – 90% RH	40% – 70% RH	< 30% or > 80% RH

C. Circuit Diagram

The circuit integrates all hardware components around the ESP32 microcontroller. The power supply provides a regulated 5 V DC output distributed to the LCD display, DHT11 sensor, and ESP32 board, while the ESP32 internally generates 3.3 V for the MAX30102 and DS18B20 sensors. I2C lines (SDA: GPIO21, SCL: GPIO22) are shared between the MAX30102 and LCD display, with each device assigned a unique I2C address. The DS18B20 communicates on a single one-wire bus (GPIO5) with a 4.7kΩ pull-up resistor. The SIM900A GSM module connects via UART and requires dedicated ground and power connections. Fig. 2 shows the complete circuit diagram.

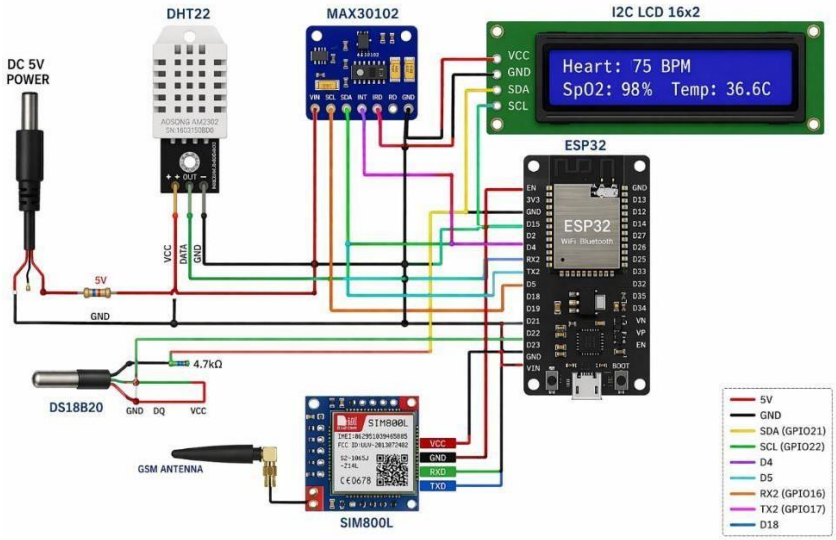


Fig. 2: Circuit Diagram of the Smart Health Monitoring System

IJARCCE ISSN (O) 2278-1021, ISSN (P) 2319-5940

International Journal of Advanced Research in Computer and Communication Engineering

Impact Factor 8.471 ⁜ Peer-reviewed & Refereed journal ⁜ Vol. 15, Issue 4, April 2026

DOI: 10.17148/IJARCCE.2026.15470

D. Software Design and Implementation

The software architecture of the system comprises two principal components: the embedded firmware running on the ESP32 and the Python-based machine learning application.

Embedded Firmware (Arduino IDE): The ESP32 is programmed using the Arduino IDE with libraries including WiFi HTTPClient.h, Wire.h, LiquidCrystal_I2C.h, DallasTemperature.h, DHT.h, MAX30105.h, and spo2_algorithm.h. On startup, the firmware initializes all sensors, establishes a Wi-Fi connection, and initializes the LCD and GSM modules. In the main loop, the system continuously reads sensor values, displays them on the LCD, and periodically transmits the data to ThingSpeak via an HTTP POST request. When abnormal health thresholds are exceeded, the firmware triggers the GSM module via AT commands (AT+CMGF=1 for SMS text mode, AT+CMGS for message dispatch) to send emergency alerts.

Machine Learning Application (Python / Streamlit): A Python application built with the Streamlit framework provides an interactive web-based user interface for health condition classification. The application operates in two modes: (i) Auto Mode, in which real-time sensor data is fetched from the ThingSpeak API (JSON format) using the Python requests library, and (ii) Manual Mode, in which users enter health parameter values directly through input widgets. The application loads a pre-trained Random Forest classifier (scikit-learn) to predict the patient's health condition. The Random Forest algorithm consists of multiple decision trees that independently analyze the input data, with the final prediction determined by majority voting, providing high accuracy and robustness against overfitting. The input feature vector includes heart rate, SpO_2, body temperature, and room temperature. A classification output of 1 indicates a Normal condition, while 0 indicates a Critical condition. The interface also generates an AI-based textual explanation summarizing the predicted health status.

E. System Workflow

Fig. 3 presents the system flowchart. The system begins with initialization of Wi-Fi, sensors, LCD, and GSM modules. The user selects Auto Mode (live IoT data from ThingSpeak) or Manual Mode (direct input). The data is forwarded to the ML model, which predicts the health condition. If the condition is Normal, the system displays a 'NORMAL' status, generates an AI explanation, and updates the ThingSpeak dashboard. If the condition is Critical, the system displays a 'CRITICAL' alert, generates an AI explanation, and immediately triggers the GSM module to send an SMS alert and initiate a voice call to the caregiver or medical professional. The monitoring loop then repeats continuously.

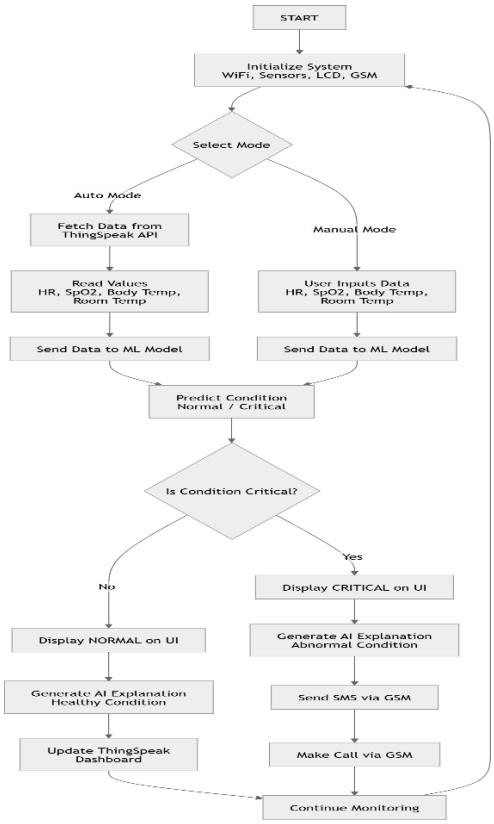


Fig. 3: System Flowchart

IJARCCE ISSN (O) 2278-1021, ISSN (P) 2319-5940

International Journal of Advanced Research in Computer and Communication Engineering

Impact Factor 8.471 ⁜ Peer-reviewed & Refereed journal ⁜ Vol. 15, Issue 4, April 2026

DOI: 10.17148/IJARCCE.2026.15470

IV. Results and Discussion

A. LCD-Based Real-Time Display

The system was tested by placing the MAX30102 sensor on the fingertip, the DS18B20 probe in contact with the skin, and the DHT11 sensor in the monitoring environment. Real-time health parameters including heart rate, SpO_2, body temperature, and room temperature were successfully displayed on the 16×2 I2C LCD. The LCD output was updated at consistent intervals, confirming stable and continuous local monitoring. Fig. 4 shows the hardware setup and the corresponding LCD output values during testing.

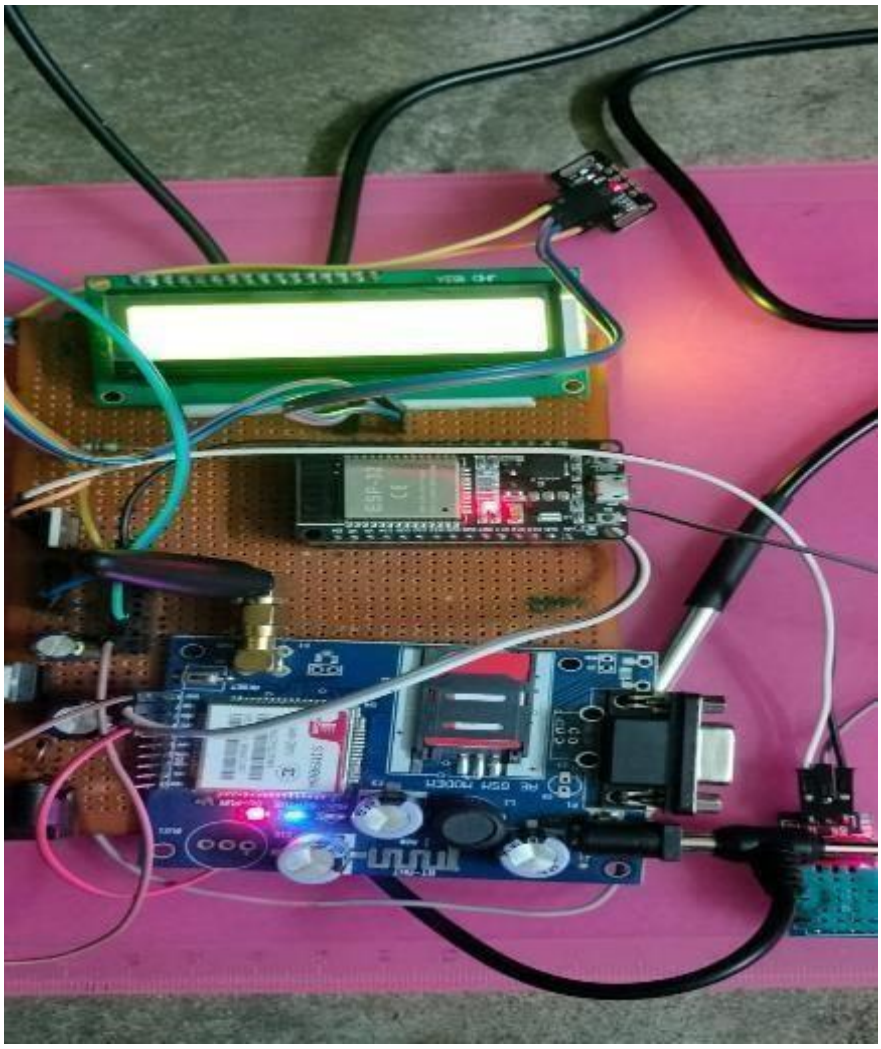


Fig. 4: Hardware Setup and LCD Output During Testing



Fig. 5: Hardware LCD Output alert

IJARCCE ISSN (O) 2278-1021, ISSN (P) 2319-5940

International Journal of Advanced Research in Computer and Communication Engineering

Impact Factor 8.471 ⁜ Peer-reviewed & Refereed journal ⁜ Vol. 15, Issue 4, April 2026

DOI: 10.17148/IJARCCE.2026.15470



Fig. 6: Hardware LCD Output

B. ThingSpeak Cloud Visualization

The sensor data was successfully uploaded to the ThingSpeak cloud platform through the ESP32 Wi-Fi module. ThingSpeak organized the data into four dedicated fields: Field 1 (Heart Rate), Field 2 (SpO_2), Field 3 (Body Temperature), and Field 4 (Room Temperature). Each field generated a real-time graphical plot, enabling trend analysis over time. Fig. 5 illustrates the ThingSpeak output, including plots for heart rate, SpO_2, and ambient humidity monitoring. The cloud storage with timestamps supports longitudinal health tracking and enables remote access by healthcare providers.

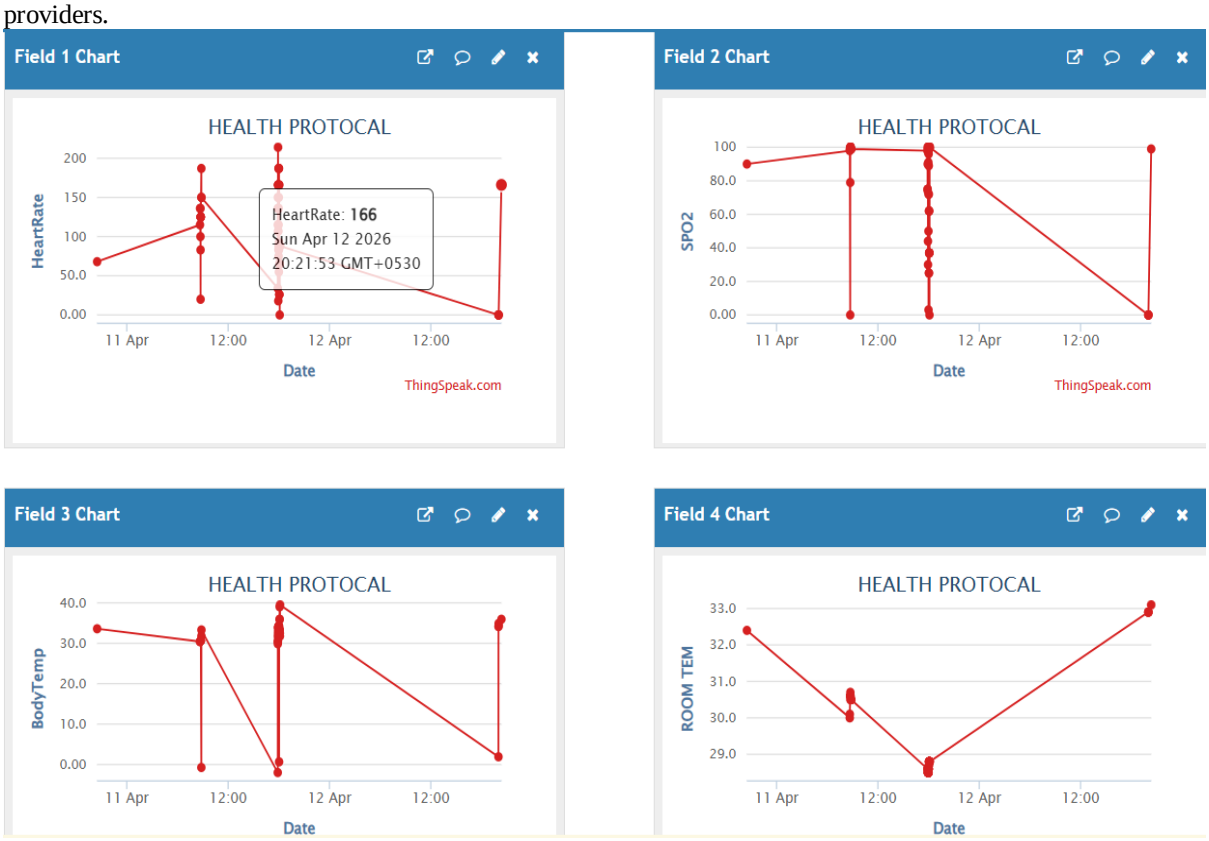


Fig. 7: ThingSpeak Cloud Output – Health Parameter Graphs

C. Machine Learning Health Classification

The Python Streamlit application was tested using both Auto Mode (live ThingSpeak API data) and Manual Mode (direct user inputs). The trained Random Forest classifier accurately classified health conditions based on the four input parameters. The model demonstrated reliable performance, correctly distinguishing Normal and Critical health states. In Auto Mode, the application fetched sensor values in JSON format from the ThingSpeak API and passed them directly to the model without manual intervention, enabling fully automated real-time health classification. Fig. 6 shows the ML model interface during patient condition checking.

IJARCCE ISSN (O) 2278-1021, ISSN (P) 2319-5940

International Journal of Advanced Research in Computer and Communication Engineering

Impact Factor 8.471 ⁜ Peer-reviewed & Refereed journal ⁜ Vol. 15, Issue 4, April 2026

DOI: 10.17148/IJARCCE.2026.15470

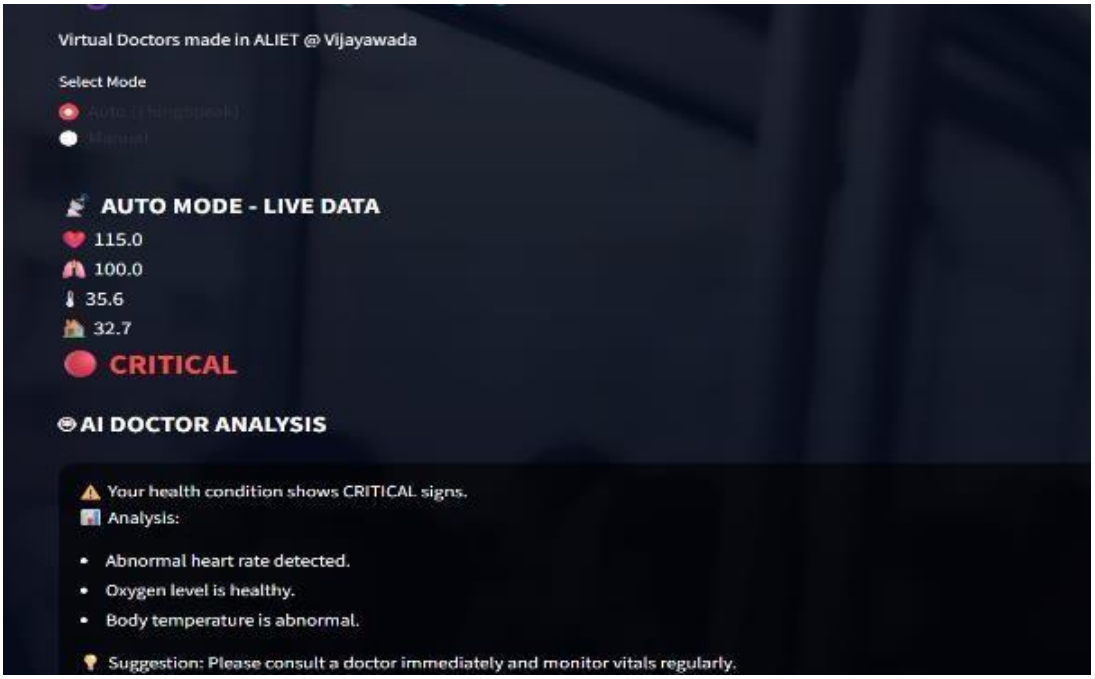


Fig. 8: ML Model Interface – Patient Condition Classification

D. GSM-Based Emergency Alert

When the ML model detected a Critical health condition, the system automatically triggered the GSM module to send an SMS alert to the pre-configured caregiver phone number. The SMS message included a notification of the abnormal health status, prompting immediate medical attention. The GSM module successfully dispatched alerts within a few seconds of critical condition detection, confirming the responsiveness of the emergency notification mechanism. Fig. 7 illustrates a sample alert SMS received during system testing.



Fig. 9: Sample Emergency Alert SMS Generated by the System

DOI: 10.17148/IJARCCE.2026.15470

E. System Advantages

The following key advantages were observed during system evaluation: (1) Real-time continuous monitoring of heart rate, SpO_2, body temperature, and room temperature; (2) Remote patient monitoring via ThingSpeak cloud, accessible from any location; (3) Intelligent health classification using a Random Forest ML model, eliminating reliance on fixed thresholds alone; (4) Automated emergency alerting through GSM SMS and voice calls for timely medical intervention; (5) Local real-time display on the LCD for immediate on-site observation; (6) Low cost and portable design suitable for home healthcare and elderly monitoring; (7) Scalability to accommodate additional sensors or advanced ML models.

F. Applications

The proposed system is applicable in a wide range of healthcare scenarios, including: home monitoring for cardiac patients, post-surgery care and hospital ward monitoring, elderly care facilities, rural and remote healthcare delivery, cardiac rehabilitation programs, integration with telemedicine platforms, and clinical research and trial data collection.

V. Conclusion

This paper presented the design, implementation, and experimental validation of an IoT and Machine Learning based Smart Health Monitoring System. The system integrates the MAX30102, DS18B20, and DHT11 sensors with an ESP32 microcontroller to continuously monitor heart rate, blood oxygen saturation, body temperature, and ambient conditions. Real-time data is displayed locally on an I2C LCD and transmitted to the ThingSpeak cloud platform for remote monitoring and trend visualization.

A Random Forest ML classifier, deployed through a Python Streamlit application, provides automated health condition classification with high reliability. The GSM-based alert mechanism ensures immediate notification to caregivers during critical health events, significantly enhancing patient safety. The system achieves its core objectives of real-time monitoring, intelligent analysis, remote accessibility, and emergency response in a single low-cost and portable platform.

The experimental results confirm that the integration of IoT and Machine Learning technologies yields a more intelligent and responsive health monitoring system compared to conventional threshold-based approaches. The proposed system offers a strong foundation for next-generation smart healthcare solutions and holds significant potential for deployment in home healthcare, elderly care, and rural medical settings.

VI. Future Work

Several avenues exist for enhancing and extending the capabilities of the proposed system. First, the ML model can be upgraded to advanced deep learning architectures such as LSTM or CNN-based classifiers to improve prediction accuracy for complex health patterns. Second, a dedicated mobile application can be developed to provide caregivers and doctors with an intuitive interface for monitoring patient health parameters from smartphones. Third, additional biomedical sensors such as ECG sensors, blood pressure sensors, and blood glucose monitoring modules can be integrated to provide a more comprehensive health assessment. Fourth, edge computing capabilities can be incorporated to enable local ML inference on the ESP32 or a companion edge device, reducing dependence on cloud connectivity and lowering response latency. Fifth, the system can be extended to support multi-patient monitoring through a centralized hospital dashboard, enabling healthcare facilities to monitor multiple patients simultaneously.

Acknowledgment

The authors express sincere gratitude to Mr. P. Bose Babu, M.Tech., (Ph.D.), Associate Professor, Department of Electronics and Communication Engineering, Andhra Loyola Institute of Engineering and Technology, for his invaluable guidance and constant encouragement throughout this work. Special thanks are extended to Dr. T. Lakshmi Narayana, Head of the Department of ECE, for his technical and moral support. The authors also acknowledge Dr. O. Mahesh, Principal of ALIET, and Rev. Fr. Dr. B. Joji Reddy S.J., Director, for providing the excellent facilities and infrastructure necessary for this research. Finally, heartfelt thanks are due to all family members for their unwavering support and encouragement.

References

[1] A. Gupta, R. Singh, and P. Kumar, "IoT-Based Smart Healthcare Monitoring System Using Wearable Sensors," in *Proc. 2023 IEEE Int. Conf. on Smart Computing and Electronic Enterprise (ICSCEE)*, IEEE, 2023.

[2] M. Khan, S. Ahmed, and T. Ali, "Real-Time IoT-Based Patient Health Monitoring System Using Cloud Computing," in *Proc. 2023 IEEE Int. Conf. on Internet of Things and Intelligence System (IoTaIS)*, IEEE, 2023.

[3] D. Patel and H. Shah, "Smart Health Monitoring System Using IoT and Machine Learning," in *Proc. 2024 IEEE Int. Conf. on Artificial Intelligence and Smart Systems (ICAISS)*, IEEE, 2024.

[4] S. Chatterjee, A. Roy, and S. Mukherjee, "AI-Enabled IoT Framework for Remote Healthcare Monitoring," *IEEE Access*, vol. 12, pp. 45520–45535, 2024. DOI: 10.1109/ACCESS.2024.3364512.

DOI: 10.17148/IJARCCE.2026.15470

[5] S. Ramesh and V. Prakash, "Intelligent IoT-Based Healthcare Monitoring System Using Edge Computing," in *Proc. 2025 IEEE Int. Conf. on Smart Internet of Things (SmartIoT)*, IEEE, 2025.

[6] G. Y. Jeong, K. H. Yu, and N. G. Kim, "Continuous Blood Pressure Monitoring Using Pulse Wave Transit Time," in *Proc. Int. Conf. on Control, Automation and Systems (ICCAS)*, 2005.

[7] K. Hung, Y. T. Zhang, and B. Tai, "Wearable Medical Devices for Tele-Home Healthcare," in *Proc. 26th Annual Int. Conf. IEEE EMBS*, 2004.

[8] W. M. Jubadi and S. F. A. M. Sahak, "Heartbeat Monitoring Alert via SMS," *Proc. 2009 IEEE Symp. on Industrial Electronics and Applications*, 2009.

[9] O. Apostu, B. Hagiu, and S. Pasca, "Wireless ECG Monitoring and Alarm System Using ZigBee," in *Proc. Int. Symp. on Advanced Topics in Electrical Engineering (ATEE)*, 2011.

[10] X. Fang et al., "An Extensible Embedded Terminal Platform for Wireless Telemonitoring," in *Proc. 2012 Int. Conf. on Information and Automation (ICIA)*, pp. 668–673. DOI: 10.1109/ICInfA.2012.6246761.

[11] G. Motika and A. Prusty, "Wireless Fetal Heartbeat Monitoring System Using ZigBee & IEEE 802.15.4 Standard," in *Proc. 2nd Int. Conf. on Emerging Applications of Information Technology*, 2011, DOI: 10.1109/EAIT.2011.89.

IJARCCE

INTERNATIONAL JOURNAL OF ADVANCED RESEARCH IN COMPUTER AND COMMUNICATION ENGINEERING

"Meets UGC Parameters"

A Monthly Peer-Reviewed & Refereed journal

CERTIFICATE OF PUBLICATION

This is to certify that

P. BOSE BABU

Assistant Professor, Department of Electronics and Communication Engineering,

Andhra Loyola Institute of Engineering and Technology (ALIET), Vijayawada, India

has published a paper entitled

IoT and Machine Learning Based Smart Health Monitoring System

Volume 15, Issue 4, April 2026

DOI: 10.17148/IJARCCE.2026.15470

ISSN (Online) 2278–1021
ISSN (Print) 2319–5940

Impact Factor 8.471
ijarcce.com

Editor-in-Chief
IJARCCE

Google Scholar doi Crossref MENDELEY

IJARCCE

INTERNATIONAL JOURNAL OF ADVANCED RESEARCH IN COMPUTER AND COMMUNICATION ENGINEERING

"Meets UGC Parameters"

A Monthly Peer-Reviewed & Refereed journal

CERTIFICATE OF PUBLICATION

This is to certify that

SIDAGAM GEETHIKA

B.Tech Student, Department of Electronics and Communication Engineering,

Andhra Loyola Institute of Engineering and Technology (ALIET), Vijayawada, India

has published a paper entitled

IoT and Machine Learning Based Smart Health Monitoring System

Volume 15, Issue 4, April 2026

DOI: 10.17148/IJARCCE.2026.15470

ISSN (Online) 2278–1021
ISSN (Print) 2319–5940

Impact Factor 8.471
ijarcce.com

Editor-in-Chief
IJARCCE

Google Scholar doi Crossref MENDELEY

CERTIFICATE OF INTERNSHIP

This is to Certify that Mr./Ms

Sidagam Geethika

Student of **Electronics And Communication Engineering**

From the college **Andhra Loyola Institute Of Engineering And Technology**

Affiliated to **Jawaharlal Nehru Technological University,Kakinada**

has successfully completed the **long-term** internship programme titled

PCB Design & Fabrication

Under SkillDzire from **15-12-2025** to **31-03-2026** , organized in collaboration with Andhra Pradesh State Council of Higher Education.

Certificate ID: **doj5g3el0h**

Issued On: **31-03-2026**

Director,
Srikanth Muppalla

Mapping of Project with PO's, PSO's and SDG's

➢ Program Outcomes(PO):

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Program Specific Outcomes (PSO):

PSO1: Develop Systems and Algorithms in the fields of VLSI, Communication, Signal Processing and Embedded Systems for contemporary needs.

PSO2: Demonstrate competence in the areas of Communication Systems through effective higher education and successful career.

CO & PO Mapping Table

Course Name	PROJECT& INDUSTRAIL INTERNSHIP	Course Code											421	
COs Code & Statement		Program Outcomes											Program Specific Outcomes	
		PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	P O 11	PSO1	PSO2
C421.1	Student will be able to apply the theoretical knowledge in practical senarios to complete the tasks assigned during the internship period.	3	2	2	2	2	1	2	2	1	2	2	2	1
C421.2	Student will be able to describe the usage of advanced tools and technologies practiced during industrial internship.	2	2	2	2	3	1	2	2	1	1	2	2	1
C421.3	Student will be able to formulate & state a research problem with technical knowledge gained during the internship period.	2	3	2	3	2	2	2	2	2				
C421.4	Student will be able to Collaborate with team members in analyzing the requirements of the project to be developed.	2	2	2	2	2	1	3	3	1	2	2	2	1
C421.5	Student will be able to build necessary design specifications and technical knowledge to implement the application	2	2	3	2	2	1	2	2					

- The 17 Sustainable Development Goals

1. **No Poverty:** End poverty in all its forms everywhere.
2. **Zero Hunger:** End hunger, achieve food security, improved nutrition, and sustainable agriculture.
3. **Good Health and Well-being:** Ensure healthy lives and promote well-being for all at all ages.
4. **Quality Education:** Ensure inclusive and equitable quality education and lifelong learning for all.
5. **Gender Equality:** Achieve gender equality and empower all women and girls.
6. **Clean Water and Sanitation:** Ensure availability and sustainable management of water and sanitation for all.
7. **Affordable and Clean Energy:** Ensure access to affordable, reliable, sustainable, and modern energy.
8. **Decent Work and Economic Growth:** Promote sustained, inclusive, and sustainable economic growth and full, productive employment.
9. **Industry, Innovation and Infrastructure:** Build resilient infrastructure, promote inclusive industrialization, and foster innovation.
10. **Reduced Inequality:** Reduce inequality within and among countries.
11. **Sustainable Cities and Communities:** Make cities inclusive, safe, resilient, and sustainable.
12. **Responsible Consumption and Production:** Ensure sustainable consumption and production patterns.
13. **Climate Action:** Take urgent action to combat climate change and its impacts.
14. **Life Below Water:** Conserve and sustainably use oceans, seas, and marine resources.
15. **Life on Land:** Protect, restore, and promote sustainable use of terrestrial ecosystems and halt biodiversity loss.
16. **Peace, Justice and Strong Institutions:** Promote peaceful, inclusive societies, provide access to justice, and build effective institutions.
17. **Partnerships for the Goals:** Strengthen implementation, including partnerships, to achieve the goals.

Project title	**SDGs Mapped**	
	SDG No.	**SDG Name**
IOT AND MACHINE LEARNING BASED SMART HEALTH MONITORING SYSTEM	3	Good Health and Well-being
	9	Industry, Innovation and Infrastructure:
	11	Sustainable Cities and Communities
	12	Responsible Consumption and Production

Signature of Guide